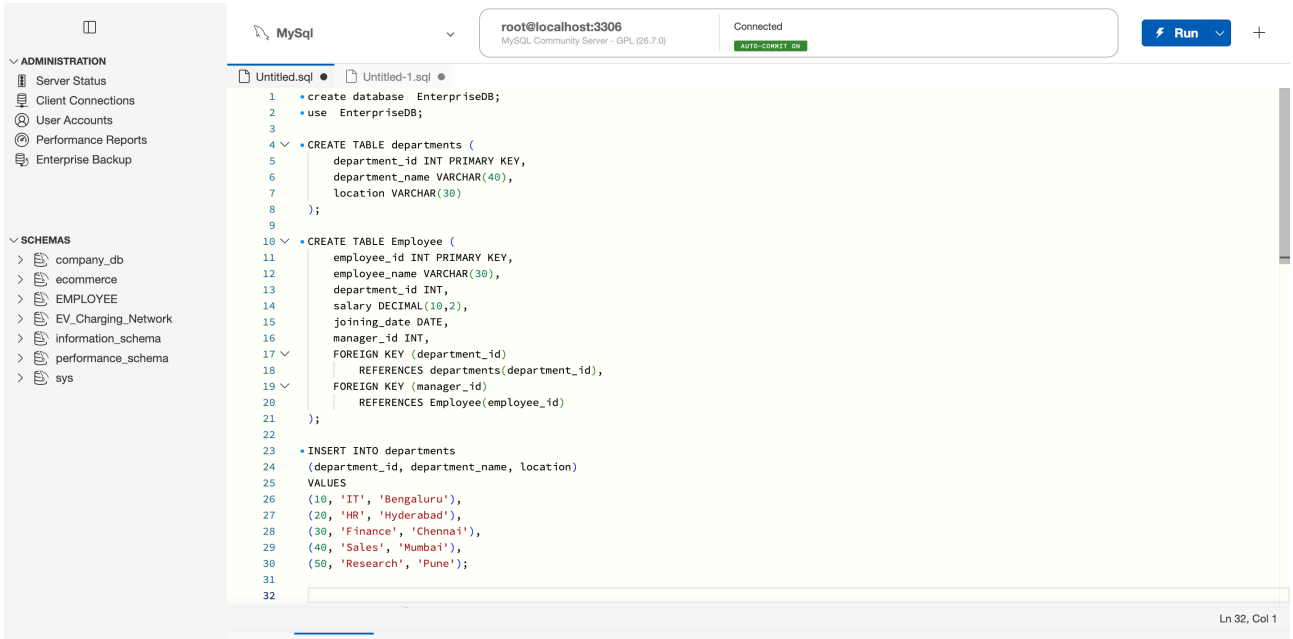
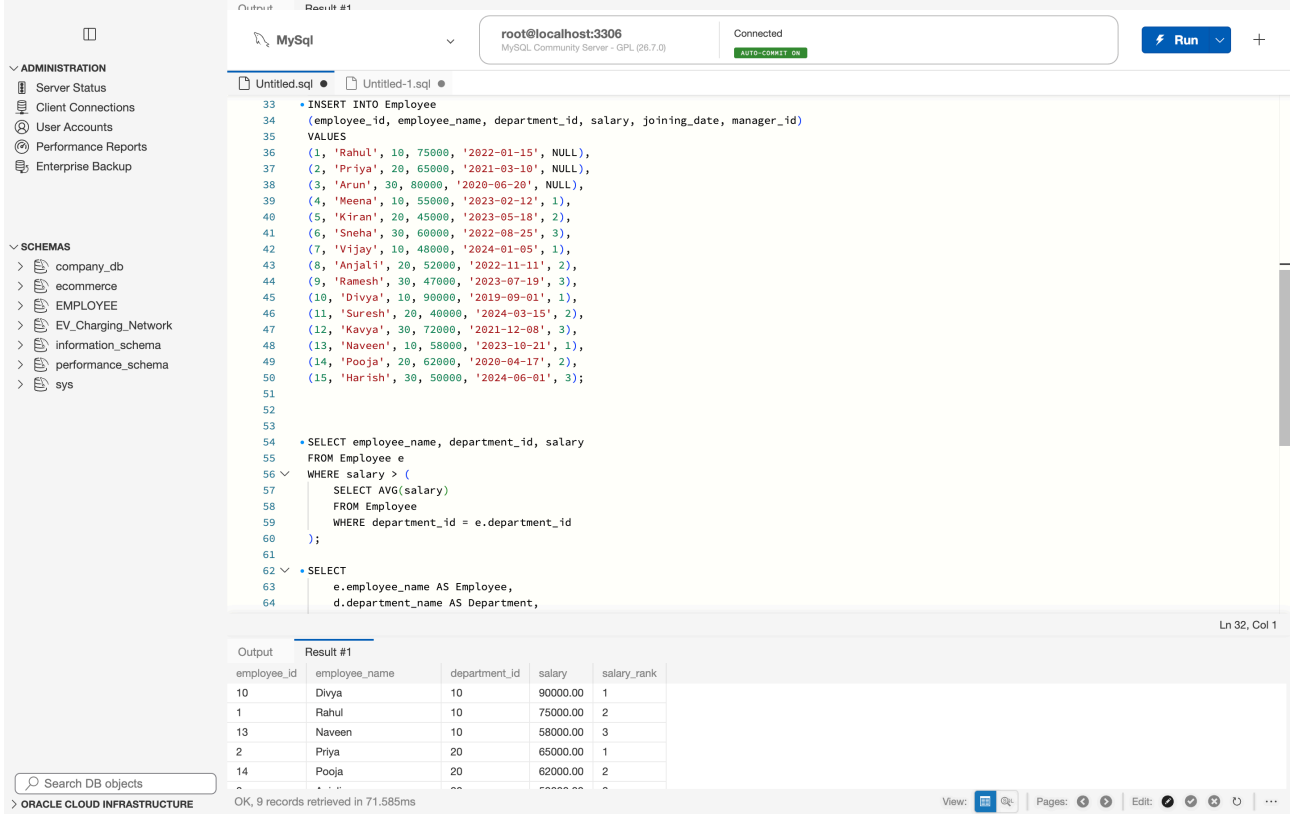
HCL ASSESSMENT - 2

NAME:KANNAIAHGARI THARUN
DEPARTMENT:CSE-IOT
USN : 23BTRCO043
HCLID: HCLTFP2264194

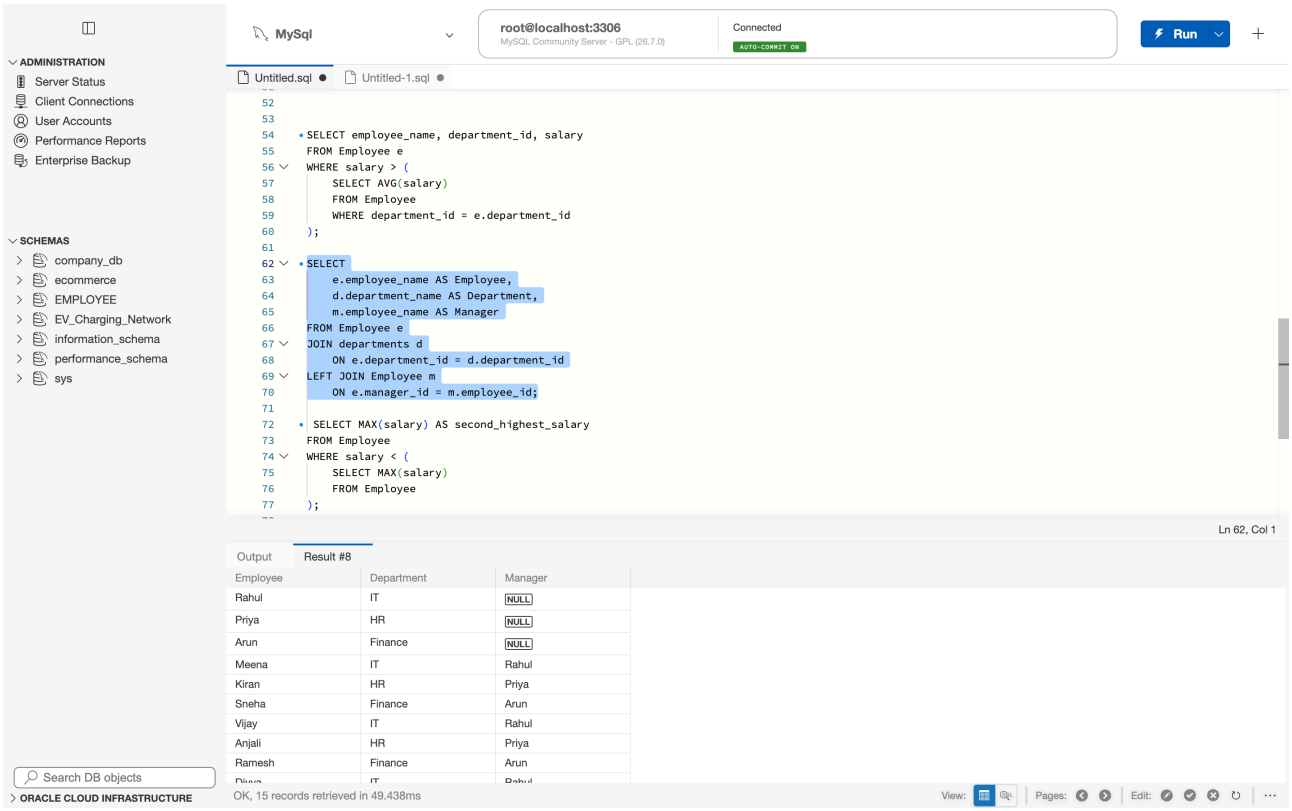
Question 1: Database Design and Advanced Queries

Create a database named EnterpriseDB.
Create the following tables.

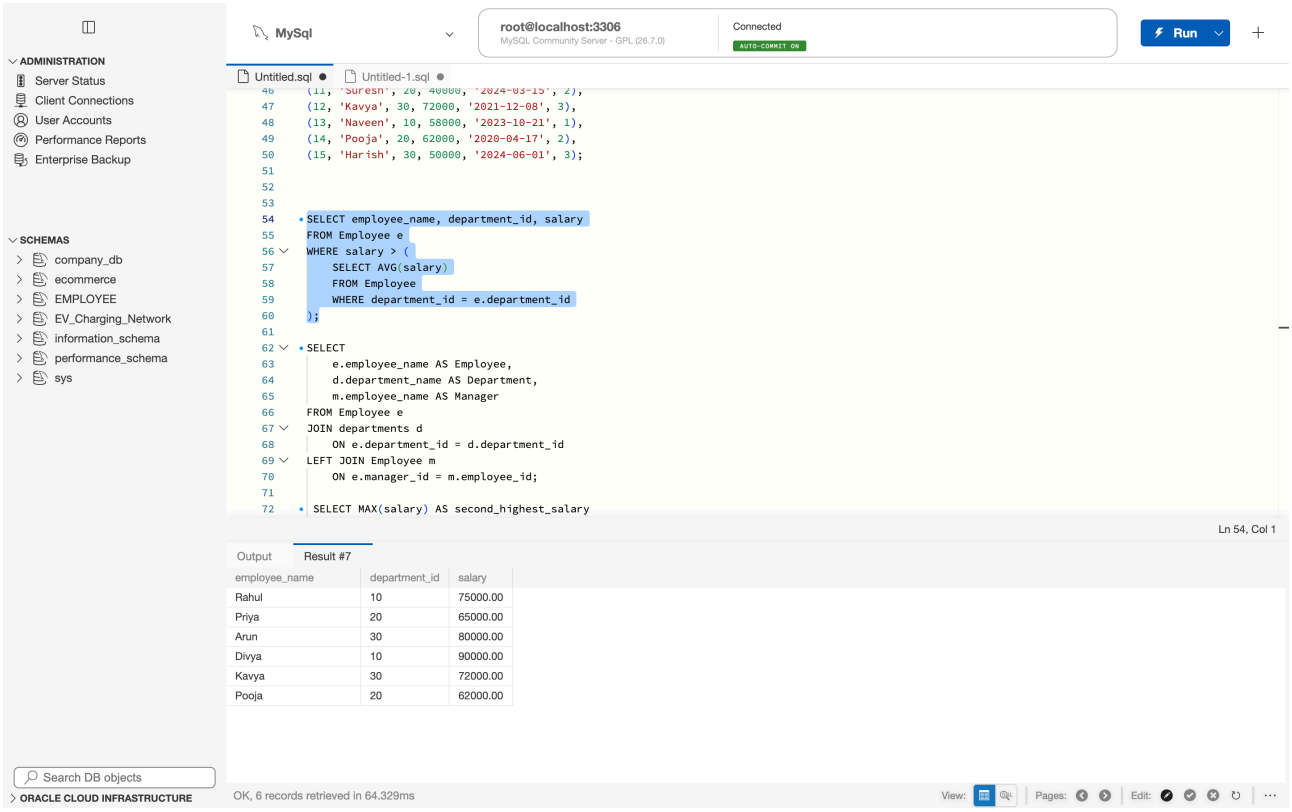




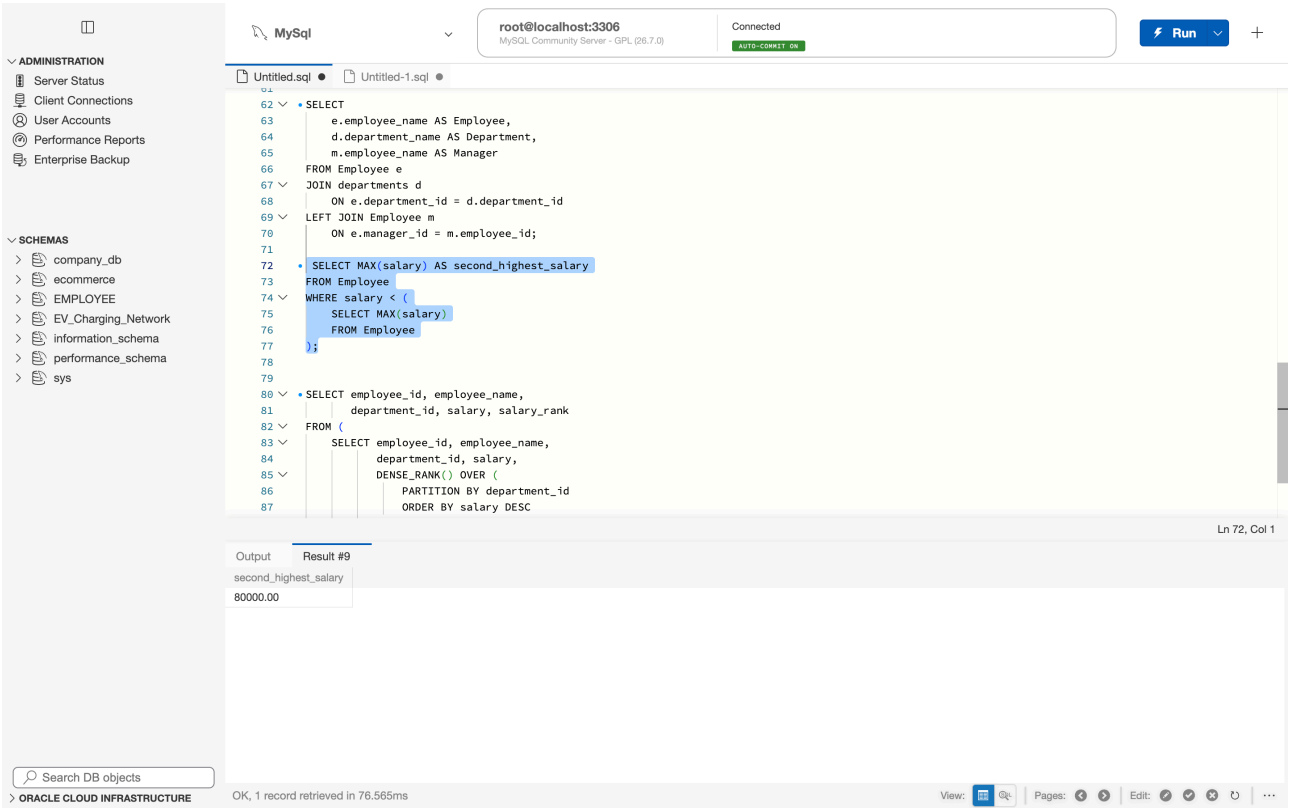
Display employee names, department names, and manager names using joins



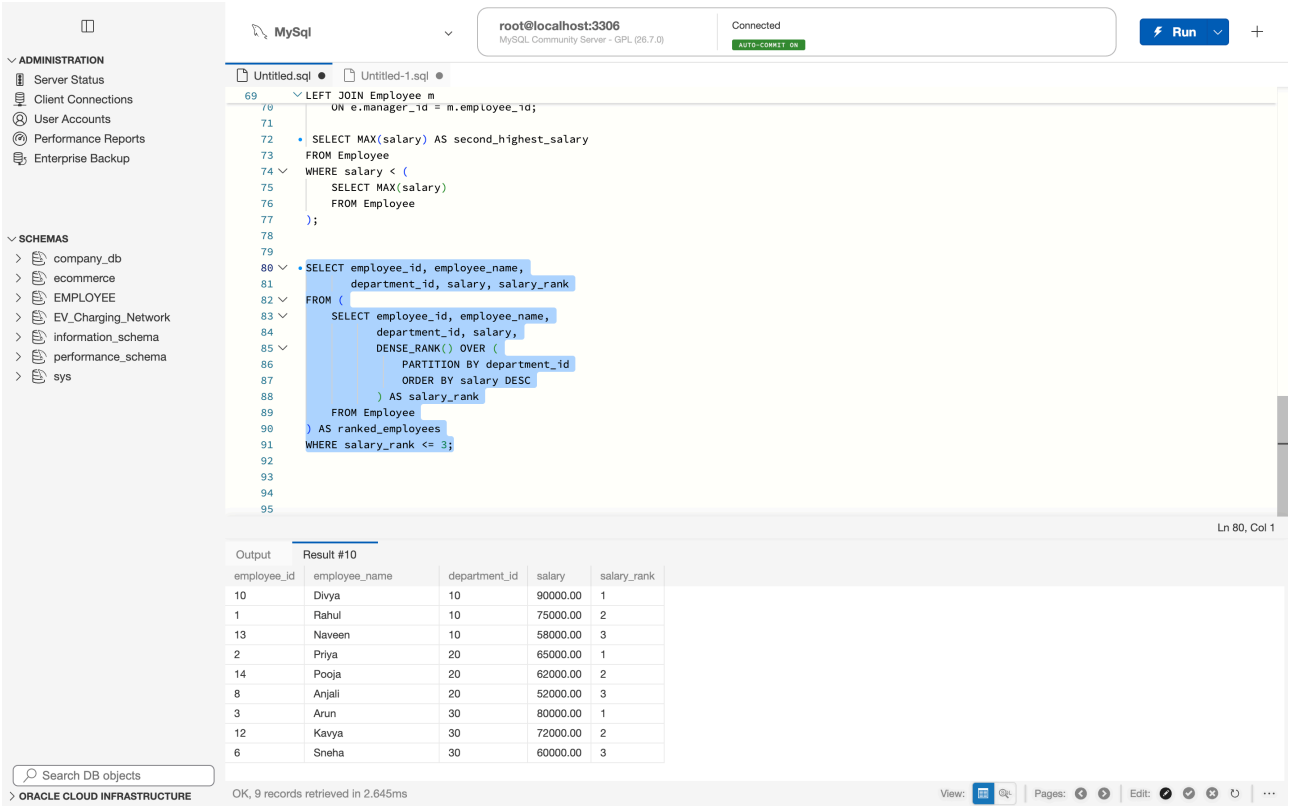
Find employees earning more than their department's average salary



Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST

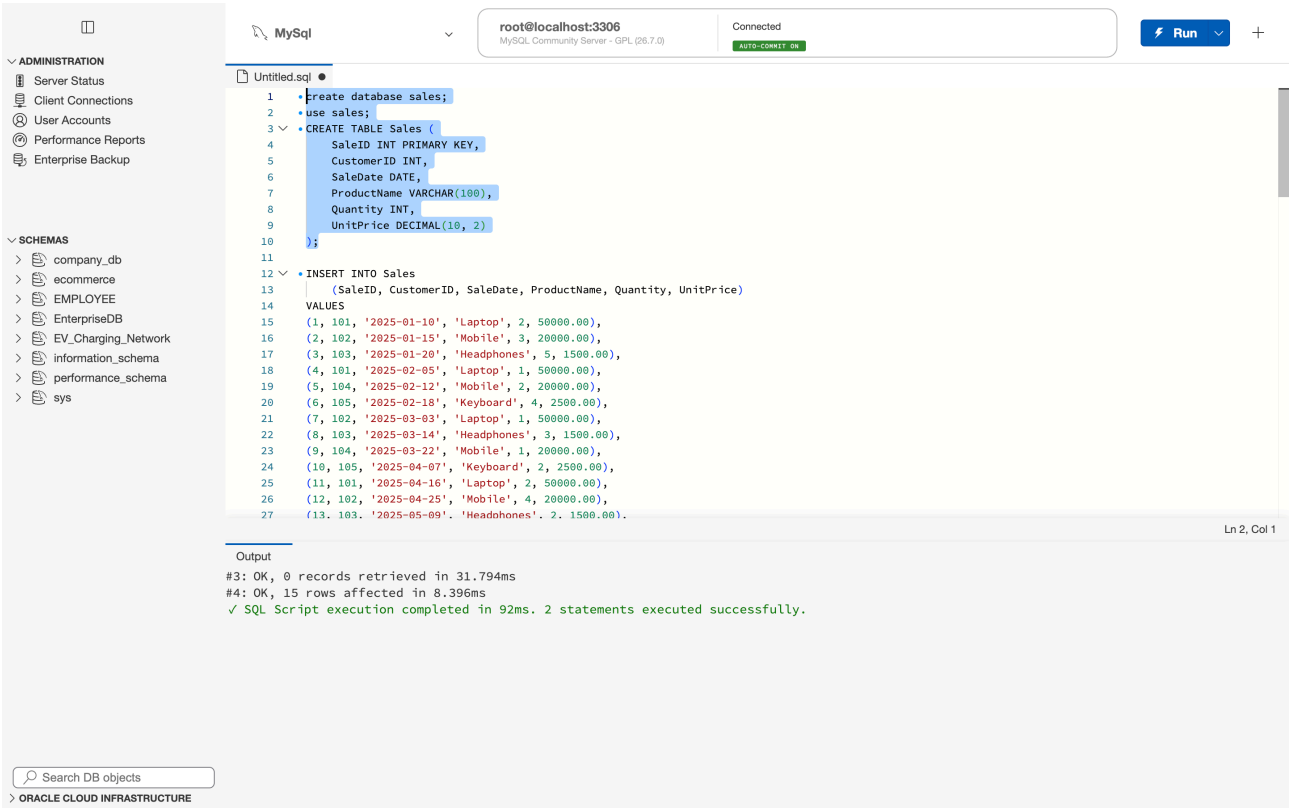


Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

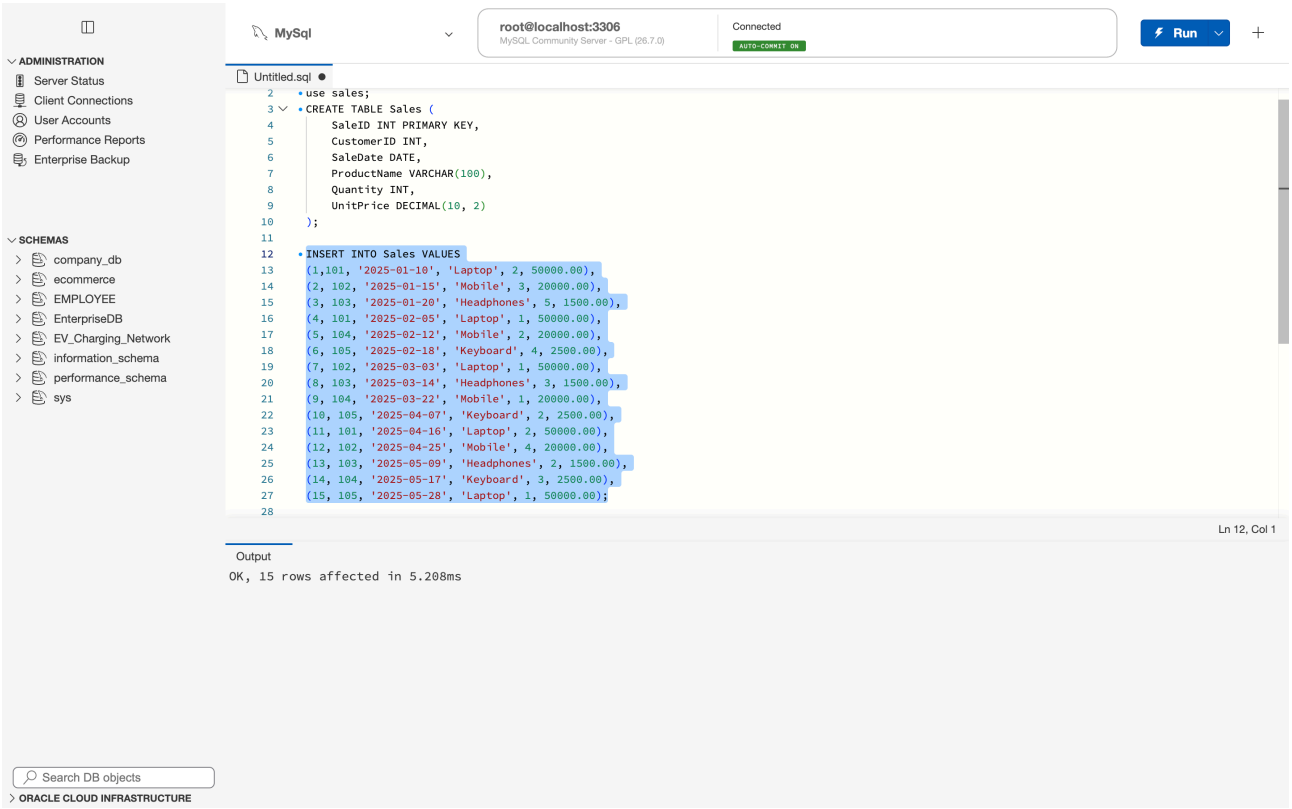


Sales Analytics and Query Optimization

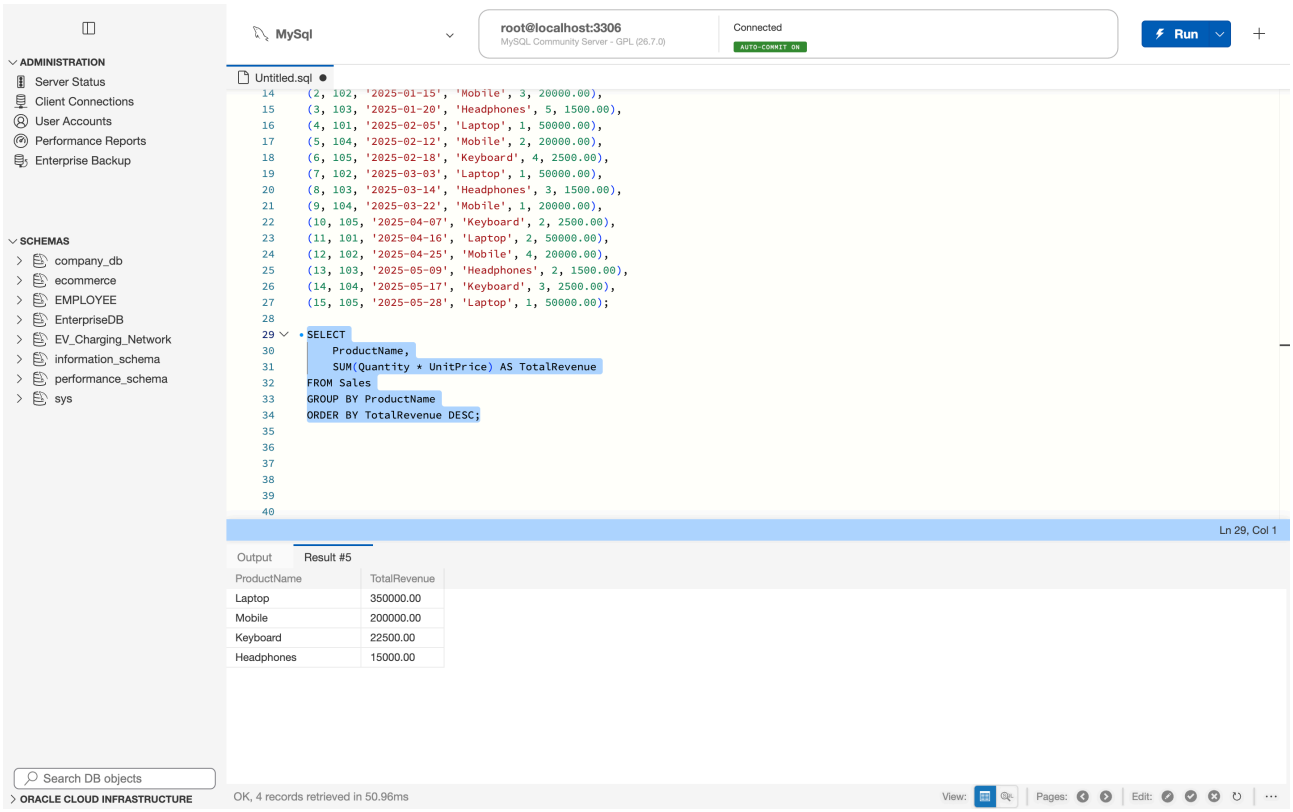
Create a Sales table with the following columns



Insert at least 15 sample records covering multiple customers, products, and dates.



Calculate total revenue for each product.



SQL Troubleshooting

a.How does an execution plan help identify expensive operations?

An execution plan shows the steps the database follows to run a query and retrieve data.
It helps find slow operations, such as scanning an entire table or performing costly joins.
It helps us understand whether indexes are being used and what changes can make the query faster.
For example, using a suitable index instead of scanning millions of records can improve query performance.

B.Recommend an index strategy for DepartmentID and Salary.

We can create a composite index on both DepartmentID and Salary.

```
CREATE INDEX idx_dept_salary
ON Employees (DepartmentID, Salary);
Which salary contain more than 50000
SELECT *
FROM Employees
WHERE DepartmentID = 10
  AND Salary > 50000;
```

C. Write a query to find duplicate customer email addresses.

```
SELECT Email, COUNT(*) AS TotalCount
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

GROUP BY Email groups customers who have the same email address.

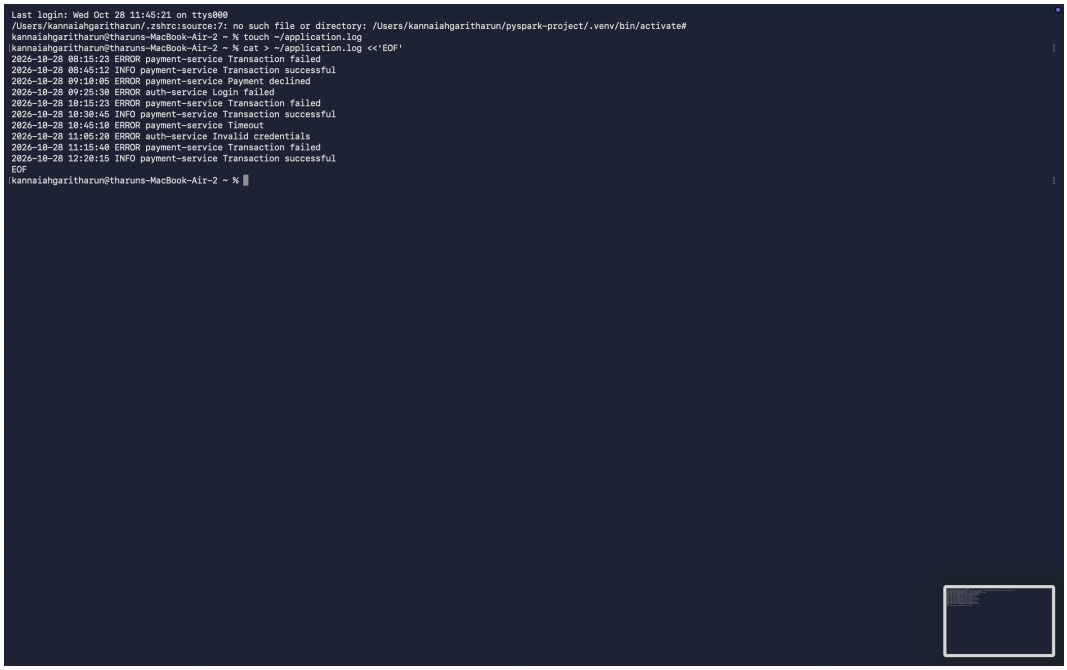
COUNT(*) counts how many times each email appears.

HAVING COUNT(*) > 1 displays only the email addresses that appear more than once

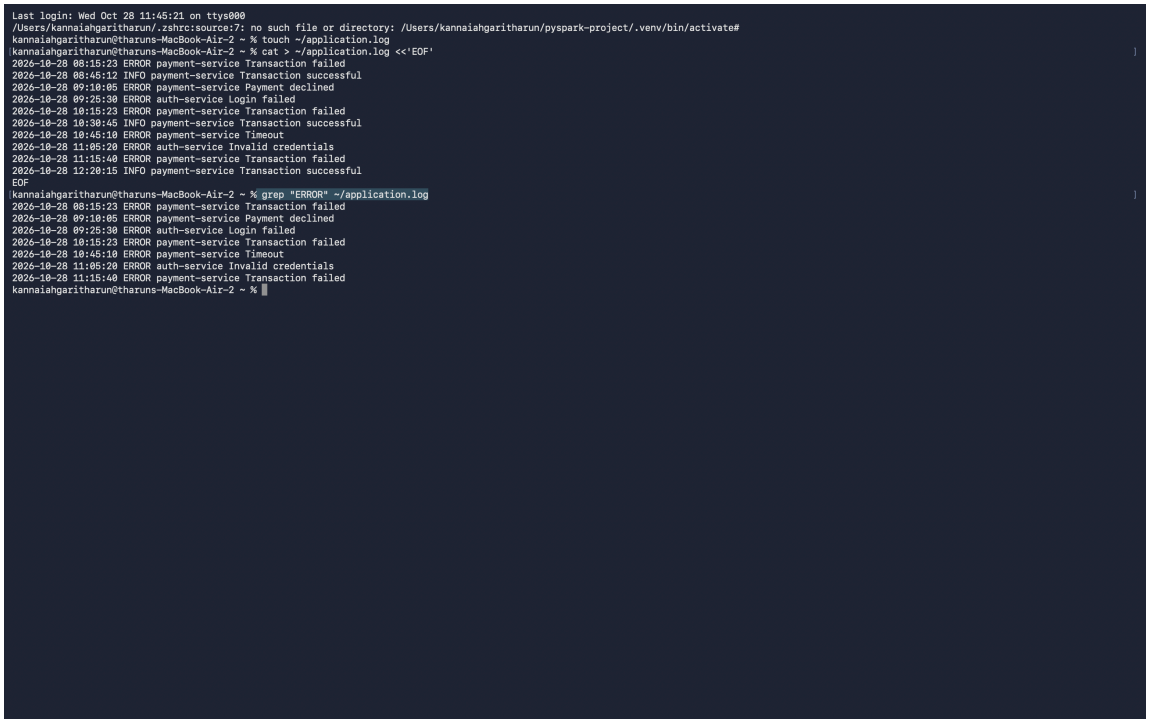
LINUX

Log Analysis

Assume /var/log/application.log contains millions of log entries in the following format:
2026-10-28 10:15:23 ERROR payment-service Transaction failed



Find all lines containing ERROR.



Display the 20 most recent error entries in the file.

```
Last login: Wed Oct 28 11:45:21 on ttys000
/Users/kannaiahgaritharun/.zshrc:source:7: no such file or directory: /Users/kannaiahgaritharun/pyspark-project/.venv/bin/activate#
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % touch ~/application.log
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cat > ~/application.log <<'EOF'
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 08:45:12 INFO payment-service Transaction successful
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:30:45 INFO payment-service Transaction successful
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
2026-10-28 12:20:15 INFO payment-service Transaction successful
EOF
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" /var/log/application.log | tail -n 20

grep: /var/log/application.log: No such file or directory
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log | tail -n 20
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ %
```

Count the total number of error entries.

```
Last login: Wed Oct 28 11:45:21 on ttys000
/Users/kannaiahgaritharun/.zshrc:source:7: no such file or directory: /Users/kannaiahgaritharun/pyspark-project/.venv/bin/activate#
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % touch ~/application.log
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cat > ~/application.log <<'EOF'
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 08:45:12 INFO payment-service Transaction successful
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:30:45 INFO payment-service Transaction successful
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
2026-10-28 12:20:15 INFO payment-service Transaction successful
EOF
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" /var/log/application.log | tail -n 20

grep: /var/log/application.log: No such file or directory
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log | tail -n 20
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep -c "ERROR" ~/application.log
7
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ %
```

Extract error entries for a specified date into a separate file.

```
Last login: Wed Oct 28 11:45:21 on ttys000
/Users/kannaiahgaritharun/.zshrc:source:7: no such file or directory: /Users/kannaiahgaritharun/pyspark-project/.venv/bin/activate#
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % touch ~/application.log
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cat > ~/application.log <<'EOF'
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 08:45:12 INFO payment-service Transaction successful
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:30:45 INFO payment-service Transaction successful
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
2026-10-28 12:20:15 INFO payment-service Transaction successful
EOF
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" /var/log/application.log | tail -n 20

grep: /var/log/application.log: No such file or directory
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log | tail -n 20
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep -c "ERROR" ~/application.log
7
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "^2026-10-28 .*ERROR" ~/application.log > ~/errors-2026-10-28.log
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cat ~/errors-2026-10-28.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ %
```

Calculate the number of errors occurring in each hour

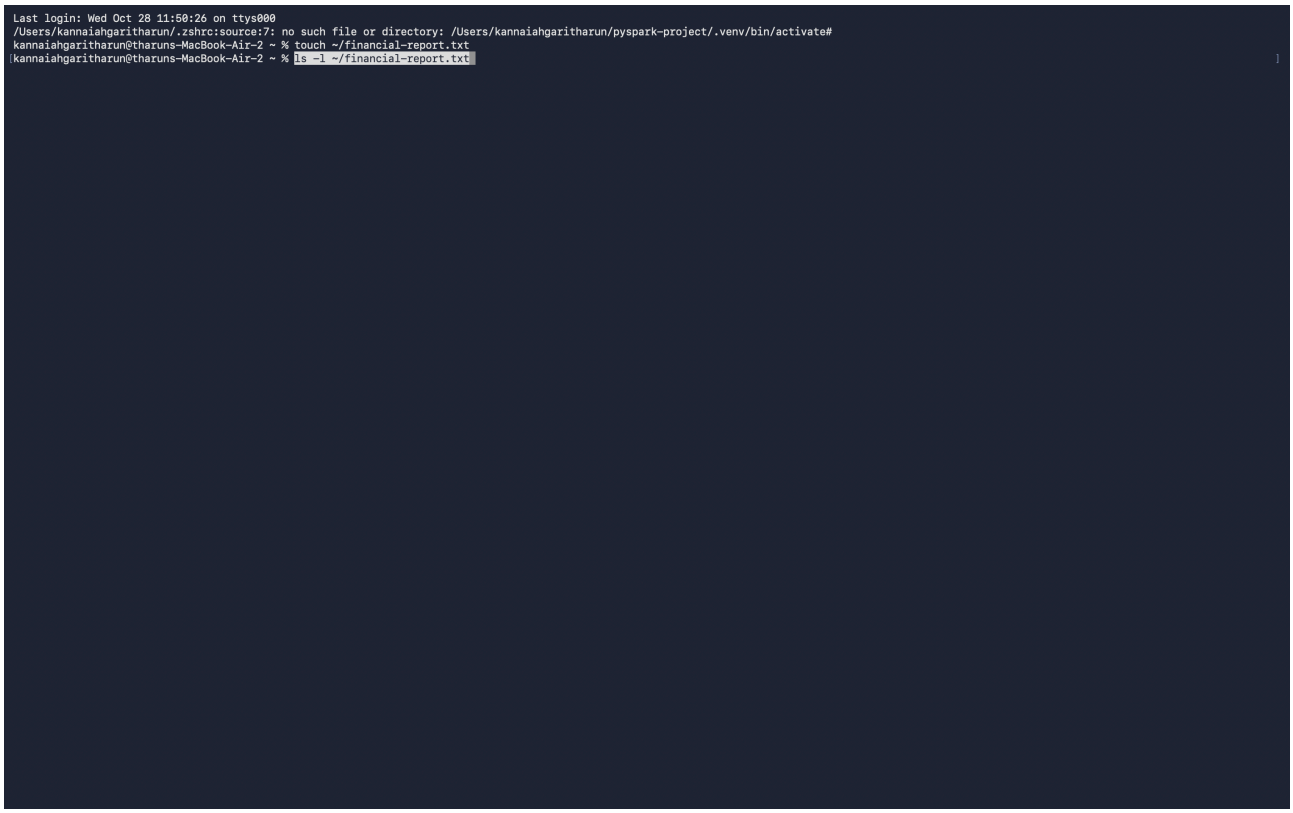
```
Last login: Wed Oct 28 11:45:21 on ttys000
/Users/kannaiahgaritharun/.zshrc:source:7: no such file or directory: /Users/kannaiahgaritharun/pyspark-project/.venv/bin/activate#
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % touch ~/application.log
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cat > ~/application.log <<'EOF'
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 08:45:12 INFO payment-service Transaction successful
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:30:45 INFO payment-service Transaction successful
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
2026-10-28 12:20:15 INFO payment-service Transaction successful
EOF
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" /var/log/application.log | tail -n 20

grep: /var/log/application.log: No such file or directory
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log | tail -n 20
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep -c "ERROR" ~/application.log
7
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "^2026-10-28 .*ERROR" ~/application.log > ~/errors-2026-10-28.log
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cat ~/errors-2026-10-28.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:10:05 ERROR payment-service Payment declined
2026-10-28 09:25:30 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:45:10 ERROR payment-service Timeout
2026-10-28 11:05:20 ERROR auth-service Invalid credentials
2026-10-28 11:15:40 ERROR payment-service Transaction failed
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % grep "ERROR" ~/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c
   1 08
   2 09
   2 10
   2 11
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ %
```

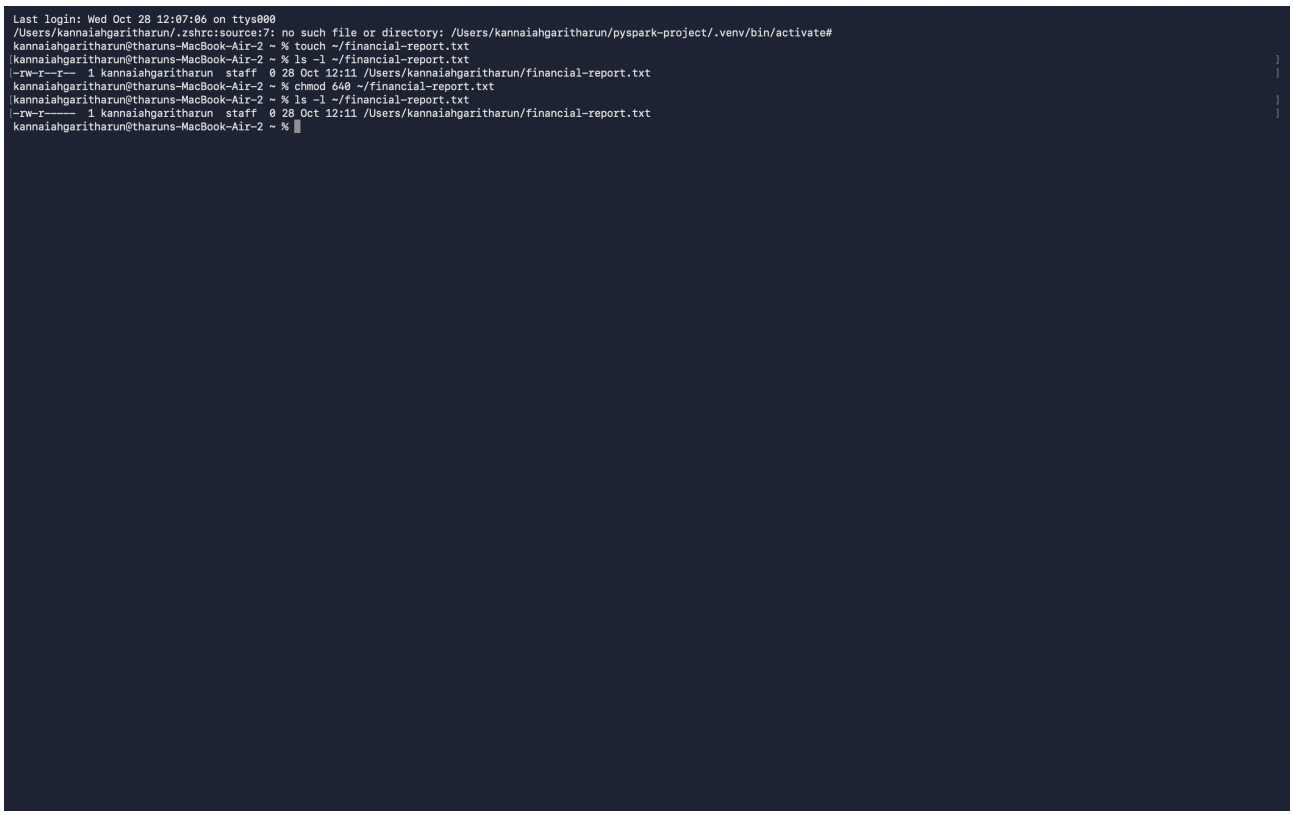
Permissions and System Troubleshooting

A Linux server hosts an application that processes confidential financial reports.

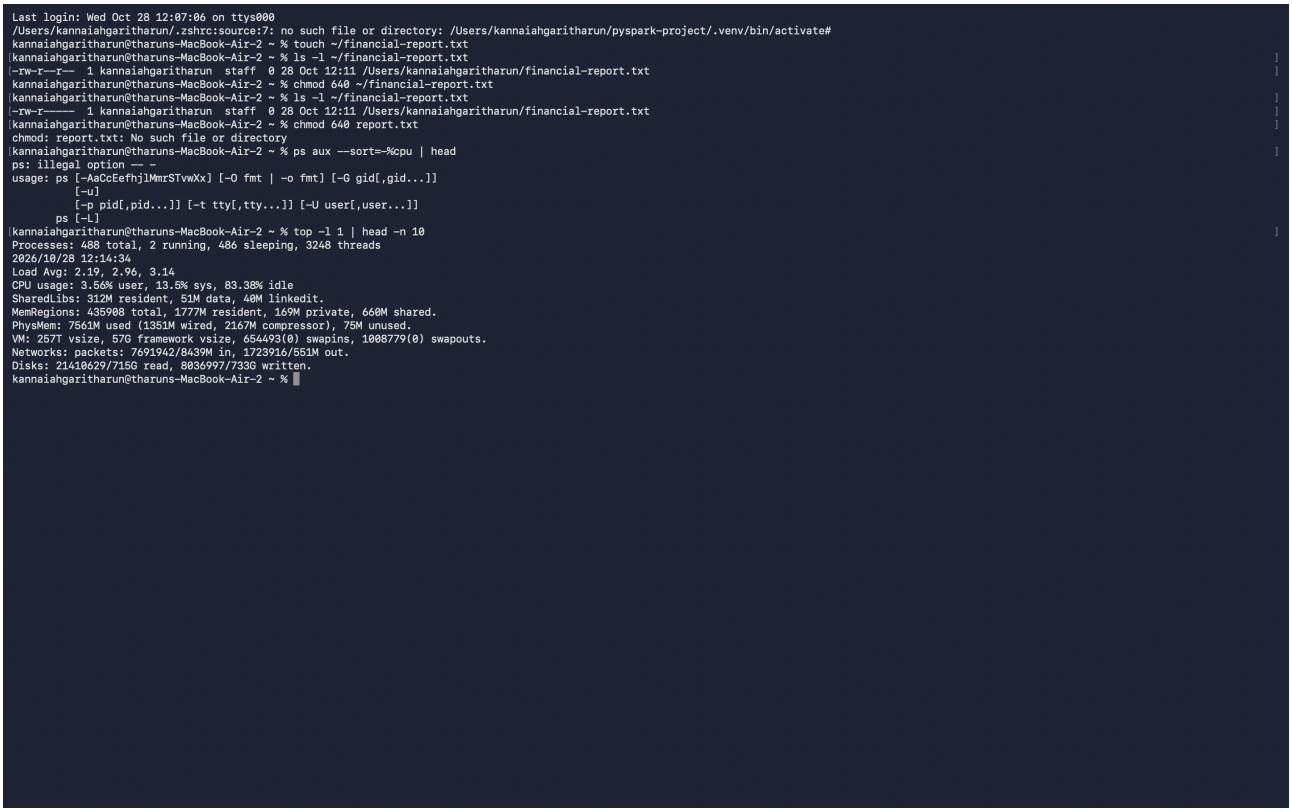
Create a group named finance and a report file, assuming you have the required privileges



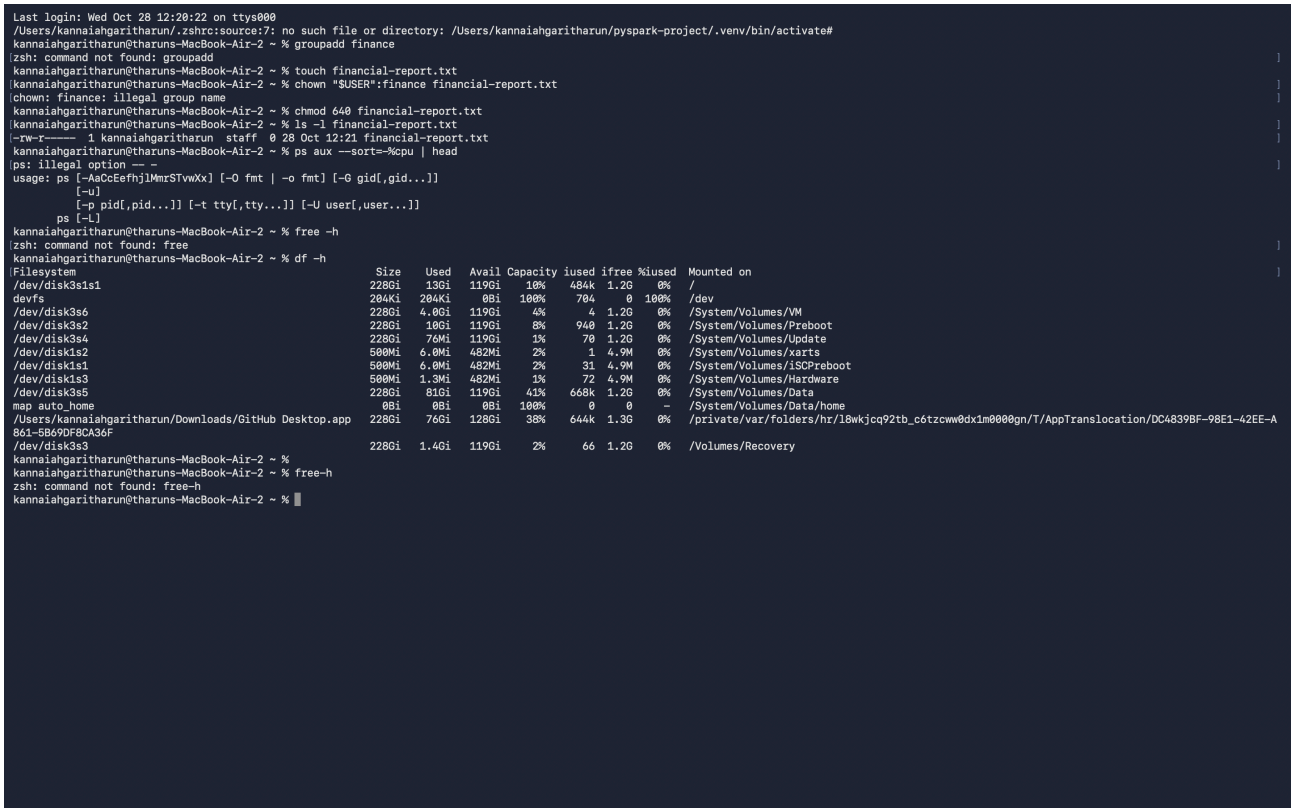
set permissions so the owner can read and write, the finance group can read, and other users have no access



Identify the processes consuming the most CPU



Display memory usage and filesystem disk utilization



Bash Scripting

Write a Bash script named backup_check.sh that:

```
Last login: Wed Oct 28 12:20:34 on ttys000
/Users/kannaiahgaritharun/.zshrc:source:7: no such file or directory: /Users/kannaiahgaritharun/pyspark-project/.venv/bin/activate#
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % mkdir backup_practice
kannaiahgaritharun@tharuns-MacBook-Air-2 ~ % cd backup_practice
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % mkdir myfiles
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % echo "Hello Backup" > myfiles/sample.txt
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % nano backup_check.sh
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % chmod +x backup_check.sh
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % ./backup_check.sh myfiles backups
Backup successful!
Backup file: backups/backup_20261028_124524.tar.gz
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % ./backup_check.sh
Error: Provide source and destination directories.
Usage: ./backup_check.sh <source> <destination>
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % ./backup_check.sh wrong_folder backups
Error: Source directory does not exist.
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % ls backups
backup_20261028_124524.tar.gz
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % tar -tzf backups/backup_*.tar.gz
myfiles/
myfiles/sample.txt
kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice %
```

Expected output:

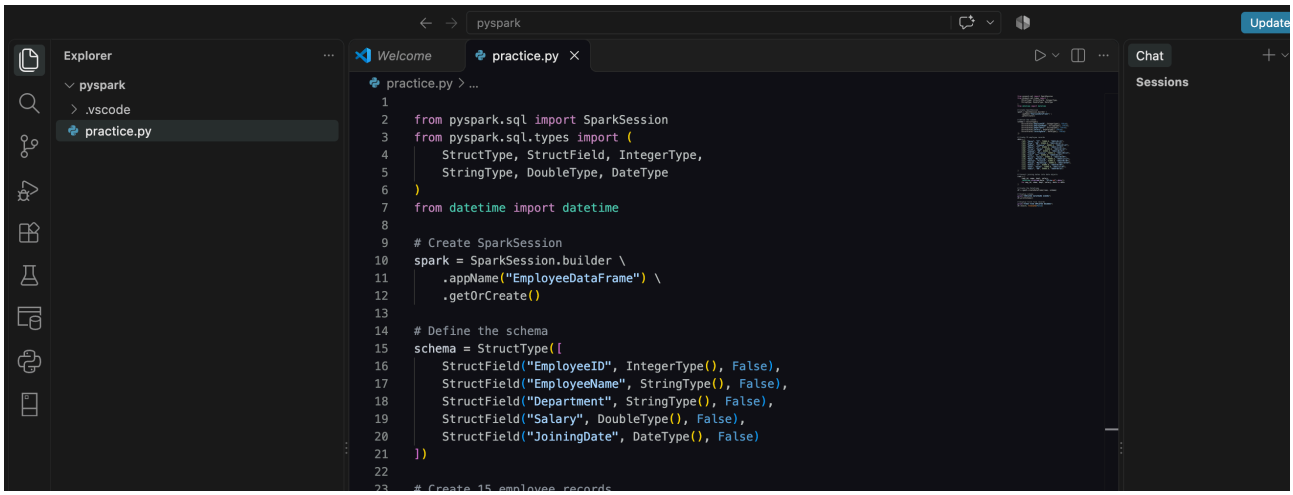
```
[kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice % tar -tzf backups/backup_*.tar.gz
 myfiles/
 myfiles/sample.txt
 kannaiahgaritharun@tharuns-MacBook-Air-2 backup_practice %
```

This script takes a source folder and a destination folder, checks whether the source folder exists, creates the destination folder if needed, and saves a compressed backup with a timestamp. It displays a success or error message depending on the result

PYSPARK DATA ENGINEERING

PySpark DataFrame Operations

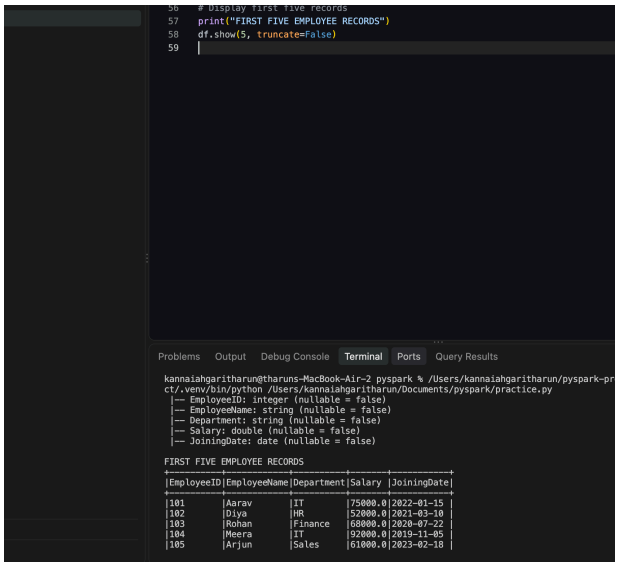
Create a PySpark DataFrame containing the following columns:
- EmployeeID
- EmployeeName
- Department
- Salary
- JoiningDate



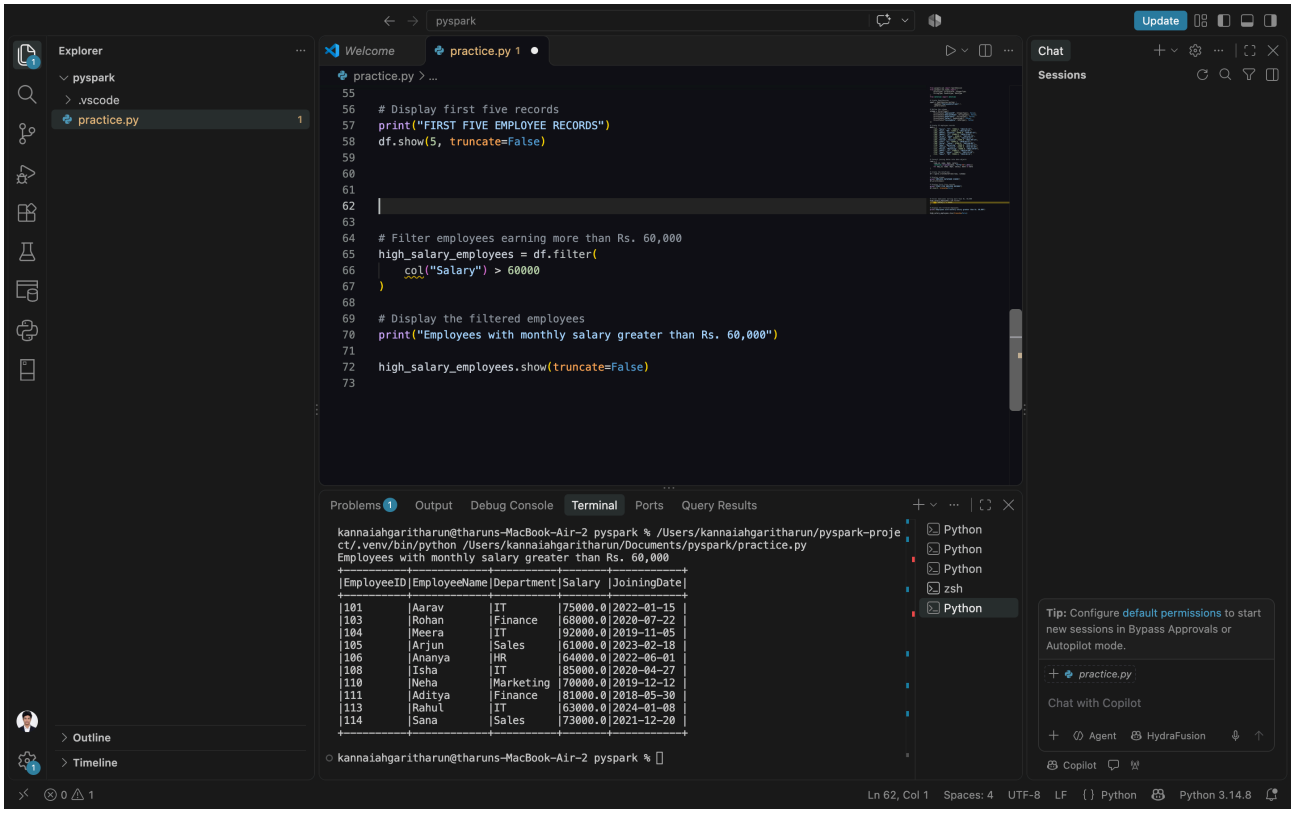
Insert at least 15 sample records.



Display the schema and first five records



Filter employees whose monthly salary is greater than ₹60,000.



```
# Display first five records
print("FIRST FIVE EMPLOYEE RECORDS")
df.show(5, truncate=False)


# Filter employees earning more than Rs. 60,000
high_salary_employees = df.filter(
    col("Salary") > 60000
)

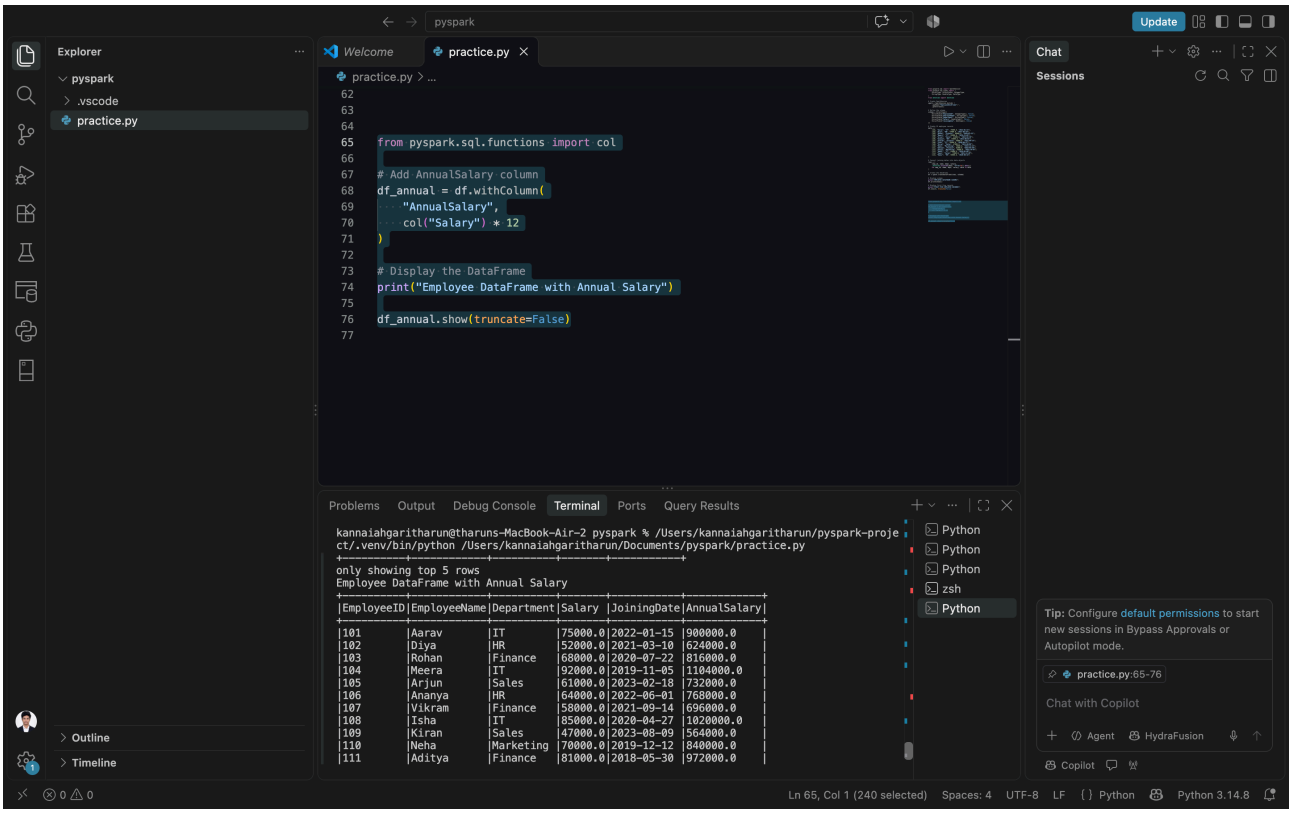
# Display the filtered employees
print("Employees with monthly salary greater than Rs. 60,000")

high_salary_employees.show(truncate=False)
```

```
kannaiahgaritharun@tharuns-MacBook-Air-2 pyspark % /Users/kannaiahgaritharun/pyspark-proje
ct/.venv/bin/python /Users/kannaiahgaritharun/Documents/pyspark/practice.py
Employees with monthly salary greater than Rs. 60,000
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Aarav       |IT        |75000.0|2022-01-15 |
|103       |Rohan       |Finance   |68000.0|2020-07-22 |
|104       |Meera       |IT        |92000.0|2019-11-05 |
|105       |Arjun       |Sales     |61000.0|2023-02-18 |
|106       |Ananya      |HR        |64000.0|2022-06-01 |
|108       |Isha        |IT        |85000.0|2020-04-27 |
|110       |Neha        |Marketing |70000.0|2019-12-12 |
|111       |Aditya      |Finance   |81000.0|2018-05-30 |
|113       |Rahul       |IT        |63000.0|2024-01-08 |
|114       |Sana        |Sales     |73000.0|2021-12-20 |
+----------+------------+----------+------+-----------+

kannaiahgaritharun@tharuns-MacBook-Air-2 pyspark %
```

Add an AnnualSalary column calculated from the monthly salary.



```
from pyspark.sql.functions import col

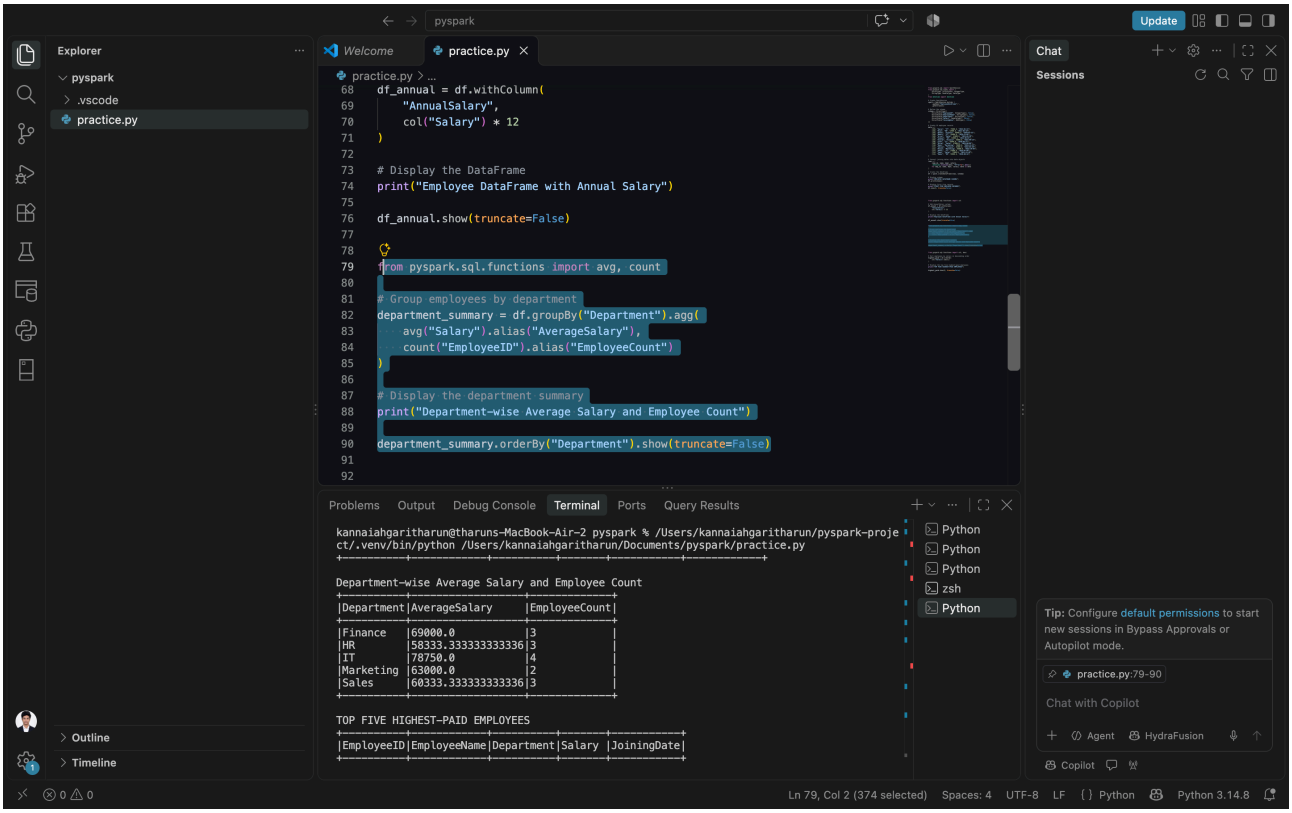
# Add AnnualSalary column
df_annual = df.withColumn(
    "AnnualSalary",
    col("Salary") * 12
)

# Display the DataFrame
print("Employee DataFrame with Annual Salary")

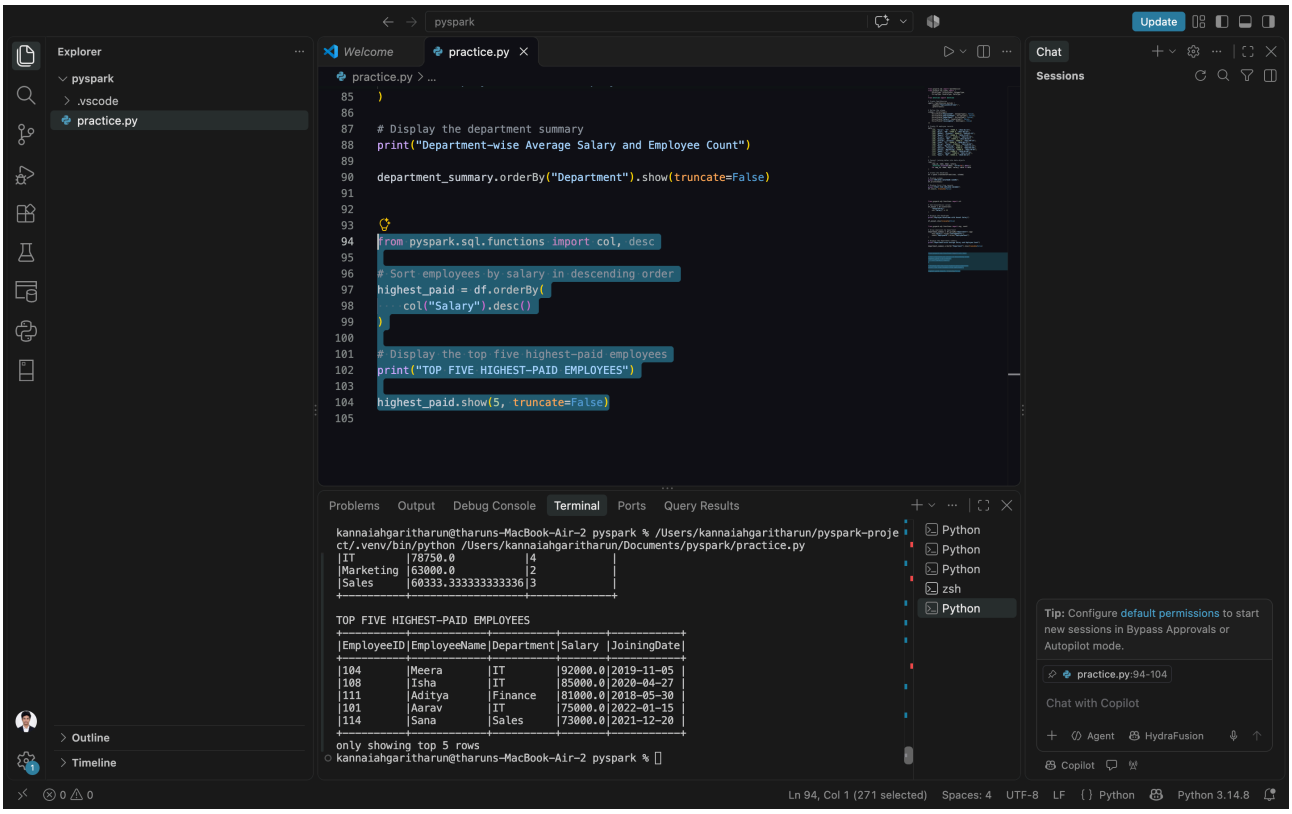
df_annual.show(truncate=False)
```

```
kannaiahgaritharun@tharuns-MacBook-Air-2 pyspark % /Users/kannaiahgaritharun/pyspark-proje
ct/.venv/bin/python /Users/kannaiahgaritharun/Documents/pyspark/practice.py
only showing top 5 rows
Employee DataFrame with Annual Salary
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|101       |Aarav       |IT        |75000.0|2022-01-15 |900000.0    |
|102       |Diya        |HR        |52000.0|2021-03-10 |624000.0    |
|103       |Rohan       |Finance   |68000.0|2020-07-22 |816000.0    |
|104       |Meera       |IT        |92000.0|2019-11-05 |1104000.0   |
|105       |Arjun       |Sales     |61000.0|2023-02-18 |732000.0    |
|106       |Ananya      |HR        |64000.0|2022-06-01 |768000.0    |
|107       |Vikram      |Finance   |58000.0|2021-09-14 |696000.0    |
|108       |Isha        |IT        |85000.0|2020-04-27 |1020000.0   |
|109       |Kiran       |Sales     |47000.0|2023-08-09 |564000.0    |
|110       |Neha        |Marketing |70000.0|2019-12-12 |840000.0    |
|111       |Aditya      |Finance   |81000.0|2018-05-30 |972000.0    |
```

Group employees by department and calculate the average salary and employee count.



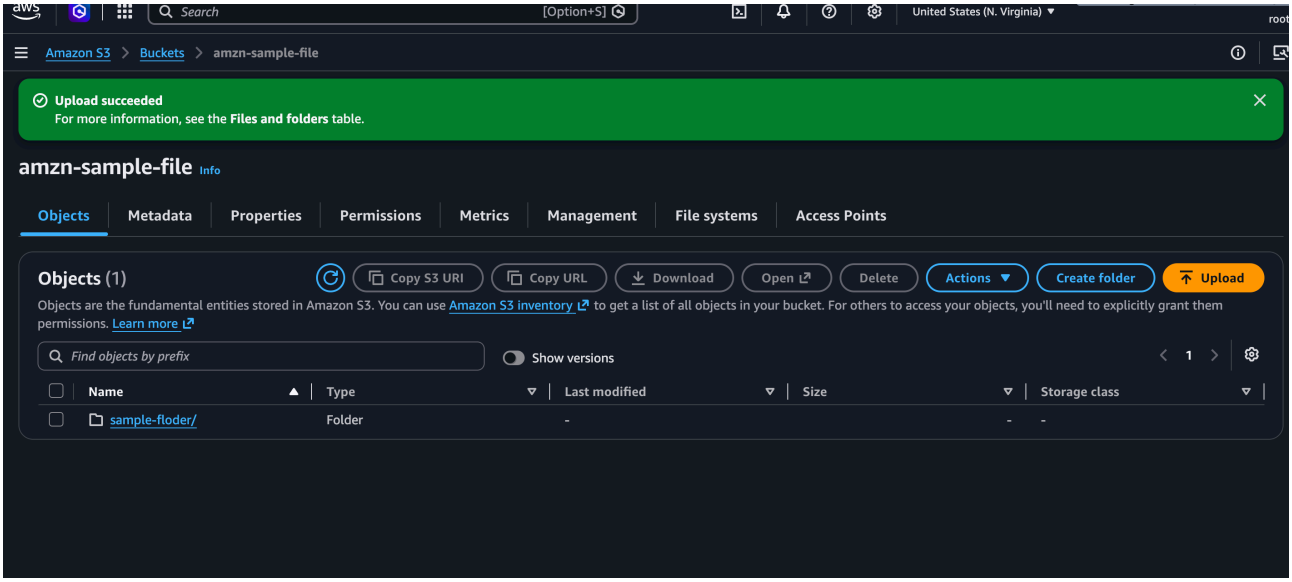
Display the five highest-paid employees



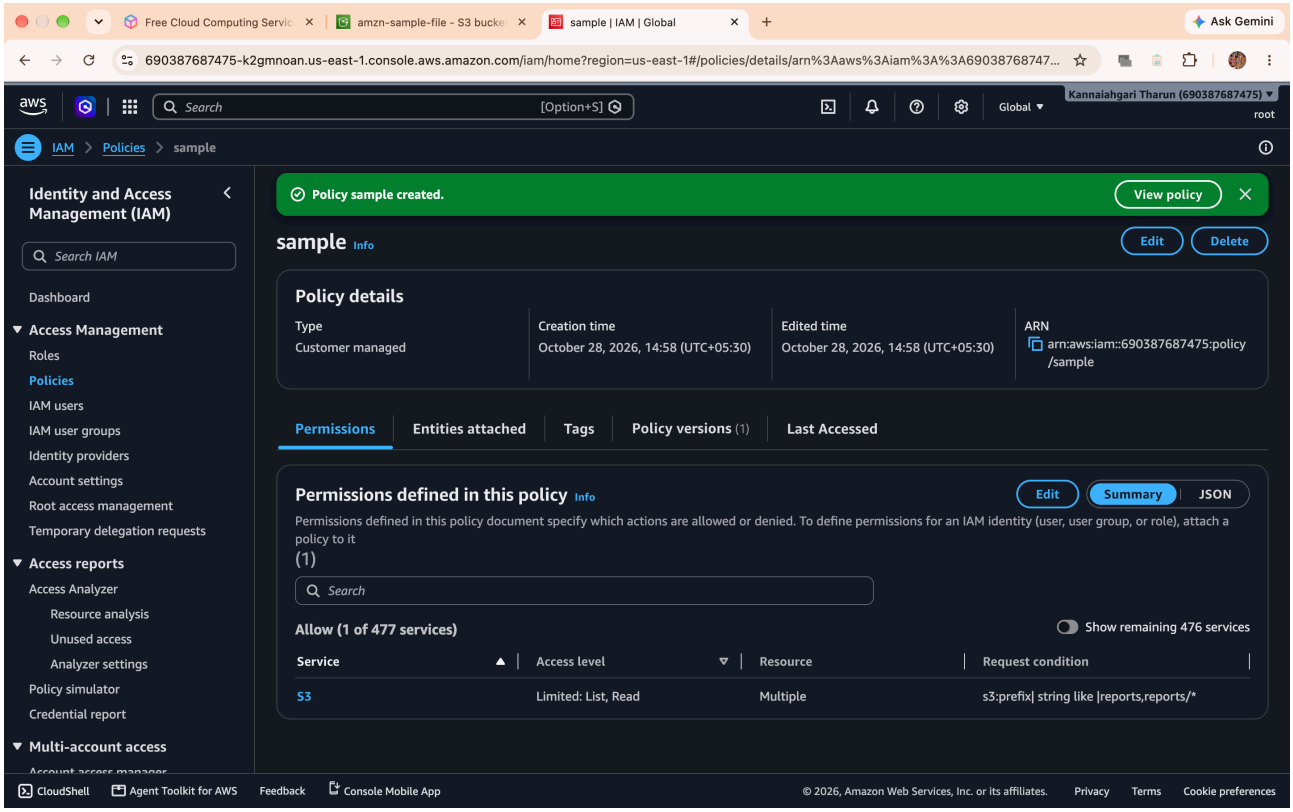
AWS CLOUD SERVICES

S3 and IAM Security

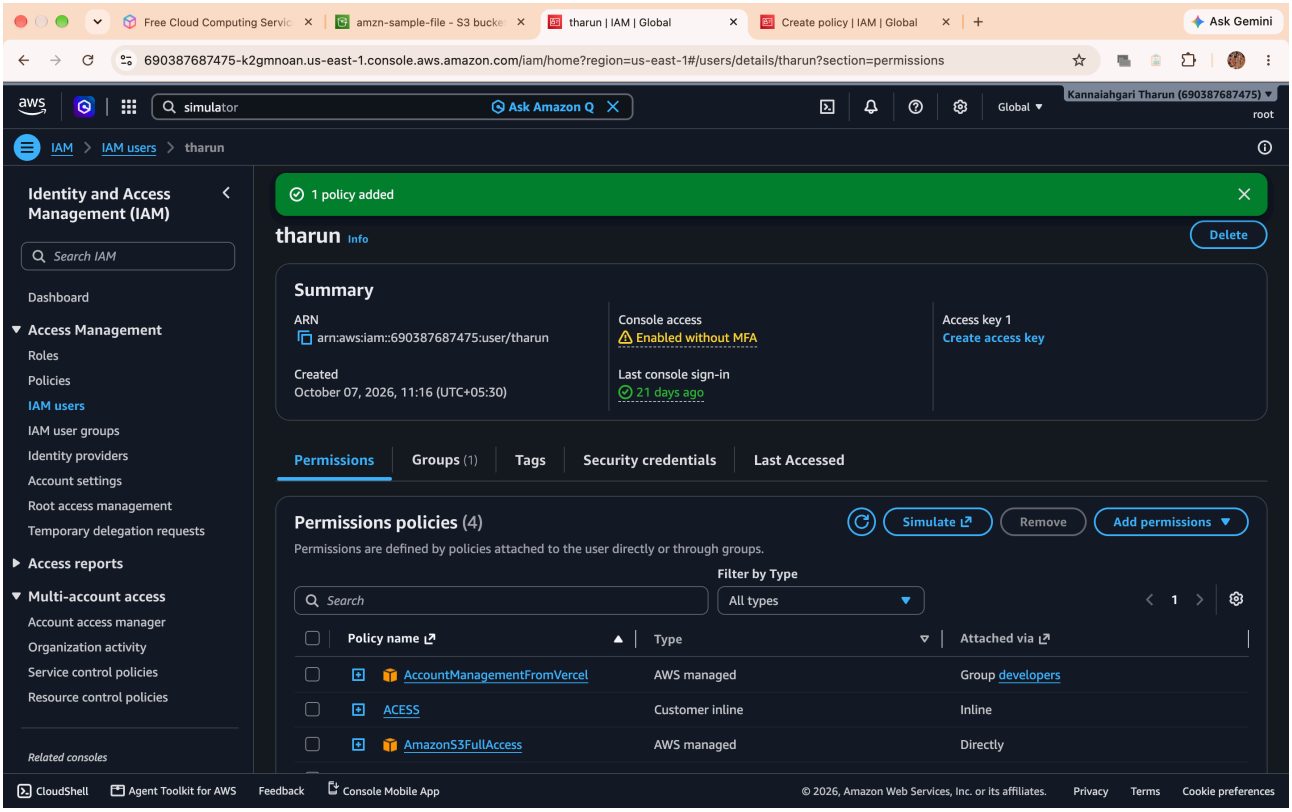
Create an S3 bucket with Block Public Access enabled and upload a sample file.



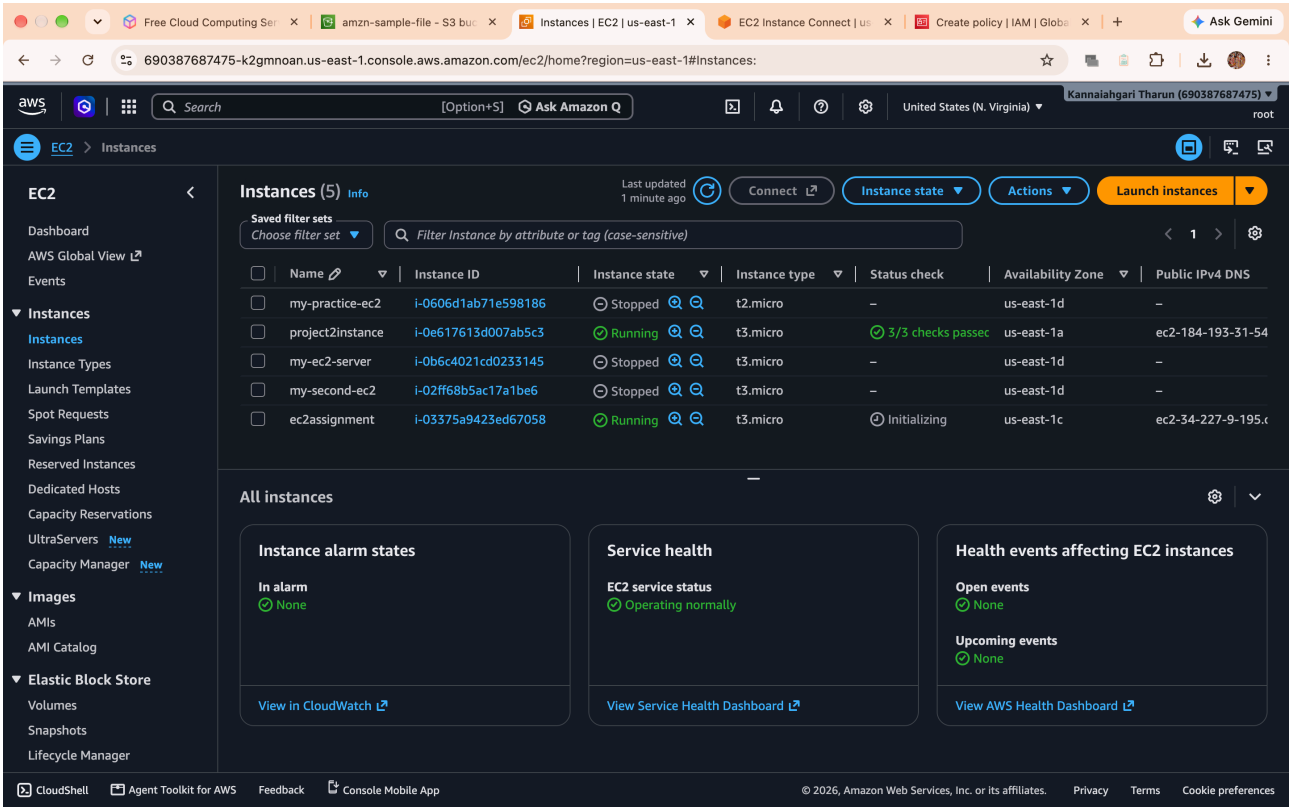
Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



Explain how you would verify that an unauthorized user cannot read the file.



EC2 Troubleshooting and CloudWatch



Explain how security groups and route tables can affect application connectivity.
A)Security groups act as virtual firewalls and control inbound and outbound traffic to EC2 instances. Missing HTTP or HTTPS inbound rules can prevent users from accessing a web application. Route tables determine how network traffic travels between subnets and gateways. Incorrect routes or a missing Internet Gateway route can cause connectivity problems.

Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period
A CloudWatch alarm was configured using the EC2 CPUUtilization metric. The statistic was set to Average with a five-minute period and a threshold greater than 85%. Two evaluation periods and two datapoints were required to trigger the alarm. This helps detect sustained high CPU utilization. An SNS notification can also be configured to alert administrators when the alarm enters the ALARM state.

AWS STEP FUNCTIONS
State Machine Design

Event-Driven Processing

Identify a suitable service for executing the processing code

ANS :
AWS Lambda is ideal for this use case as it scales automatically, requires no server management, and only charges for the compute time used.

b. Explain how an S3 upload can trigger processing.
ANS:
S3 Event Notifications can be configured on the bucket to send an event to Lambda whenever an ObjectCreated event occurs (e.g., a file is uploaded).

c. Explain the IAM permissions needed to read the input file and write the results.
ANS :
The Lambda execution role needs s3:GetObject for the source bucket and s3:PutObject for the destination bucket.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["s3:GetObject"],
            "Resource": "arn:aws:s3:::source-csv-bucket/*"
        },
        {
            "Effect": "Allow",
            "Action": ["s3:PutObject"],
            "Resource": "arn:aws:s3:::destination-processed-bucket/*"
        },
        {
            "Effect": "Allow",
            "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "l
            "Resource": "arn:aws:logs:*:*:*"
        }
    ]
}
```

This policy grants the necessary permissions for the Lambda function to read the input, write the output, and write logs to CloudWatch.

d. Explain how errors and processing logs can be captured.
ANS :
Amazon CloudWatch Logs automatically captures standard output and errors from the Lambda function.

e. Explain how to prevent duplicate processing or recursive triggering.
ANS :
Write the processed output to a different S3 bucket (or a different folder prefix) than the input bucket to prevent the output from triggering the Lambda function again.

State Machine Design

To coordinate the sales file processing workflow, you can design an AWS Step Functions state machine using Task states for validation, transformation, storage, and notification.

```JSON
{
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Next": "CheckValidation"
    },
    "CheckValidation": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.Payload.isValid",
          "BooleanEquals": true,
          "Next": "TransformData"
        }
      ],
      "Default": "ValidationFailed"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Next": "CalculateSales"
    },
    "CalculateSales": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Next": "StoreResults"
    },
    "StoreResults": {
      "Type": "Task",
      "Resource": "arn:aws:states:::aws-sdk:s3:putObject",
      "Next": "SendNotification"
    },
    "SendNotification": {
      "Type": "Task",
      "Resource": "arn:aws:states:::sns:publish",
      "End": true
    },
    "ValidationFailed": {
      "Type": "Fail",
      "Error": "InvalidFile",
      "Cause": "The input file failed validation checks."
    }
  }
}
```

Identify suitable states for validation, processing, storage, and notification.
ANS :
Use Task states for invoking Lambda (Validation, Transform, Calculate), an S3 integration for storage, and an SNS integration for the notification.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.
ANS :
The Execution History in the AWS Console shows the exact input, output, and error stack trace for every step, making it easy to pinpoint where a failure occurred.

c. Explain how Retry and Catch can handle transient failures and permanent errors.
ANS :
The Choice state checks a variable (e.g., $.isValid) from the validation step. If true, it routes to "TransformData"; if false, it routes to a "ValidationFailed" Fail state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.
ANS :
Step Functions invokes services using ARNs in the Resource field of a Task state (e.g., arn:aws:states:::lambda:invoke).

e. Explain how execution history helps troubleshoot a failed workflow.
ANS :
Retry automatically re-runs a state on transient errors (e.g., timeouts) with backoff. Catch intercepts permanent errors and routes the workflow to a fallback path (like an alert).

Step Functions Implementation

Here is the Amazon States Language (ASL) JSON definition for your sequential workflow. It includes the three required tasks, a retry policy for the transformation step, and a catch block for the loading step.

```
{
  "Comment": "Sequential Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:TransformData",
      "Retry": [
        {
          "ErrorEquals": ["States.TaskFailed", "States.Timeout"],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2.0
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:LoadResults",
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "HandleFailure"
        }
      ],
      "End": true
    },
    "HandleFailure": {
      "Type": "Fail",
      "Error": "LoadError",
      "Cause": "The workflow failed during the LoadResults stage."
    }
  }
}
```

Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.
ANS:

```
"Comment": "Sequential Data Processing Workflow",
"StartAt": "ValidateFile",
"States": {
  "ValidateFile": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:ValidateFile",
    "Next": "TransformData"
  },
  "TransformData": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:TransformData",
    "Retry": [
      {
        "ErrorEquals": ["States.TaskFailed", "States.Timeout"],
        "IntervalSeconds": 2,
        "MaxAttempts": 3,
        "BackoffRate": 2.0
      }
    ],
    "Next": "LoadResults"
  },
  "LoadResults": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:LoadResults",
    "Catch": [
      {
        "ErrorEquals": ["States.ALL"],
        "Next": "HandleFailure"
      }
```

Configure a Catch transition for the loading task that routes to an error-handling state.
ANS :

```
  },
  "HandleFailure": {
    "Type": "Fail",
    "Error": "LoadError",
    "Cause": "The workflow failed during the LoadResults stage."
  }
}
```

d. Explain how data from one task is passed to the next task.
ANS :
Data Passing: Step Functions passes data as a JSON payload. The output of ValidateFile automatically becomes the input for TransformData, unless modified using ResultPath or OutputPath.

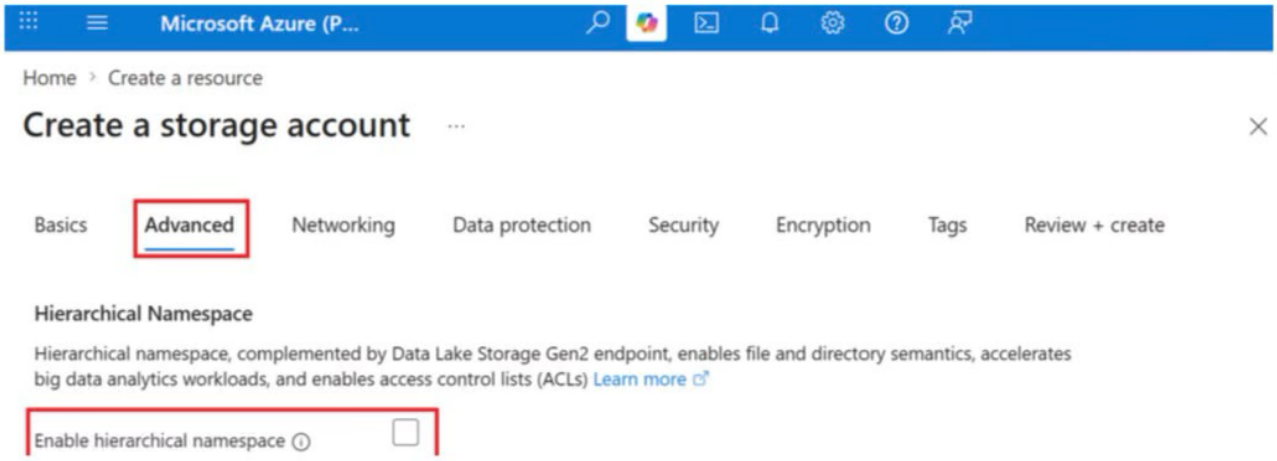
e. Explain how you would test successful and failed executions.
ANS :
Successful Execution: Start an execution with a valid mock JSON input and verify that all states turn green in the AWS Console.
Failed Execution: Pass an invalid input (or configure a mock Lambda to throw an error) to verify that the Retry logic triggers and that the Catch block successfully routes to the HandleFailure state

MICROSOFT AZURE
Azure Data Lake Storage and Security

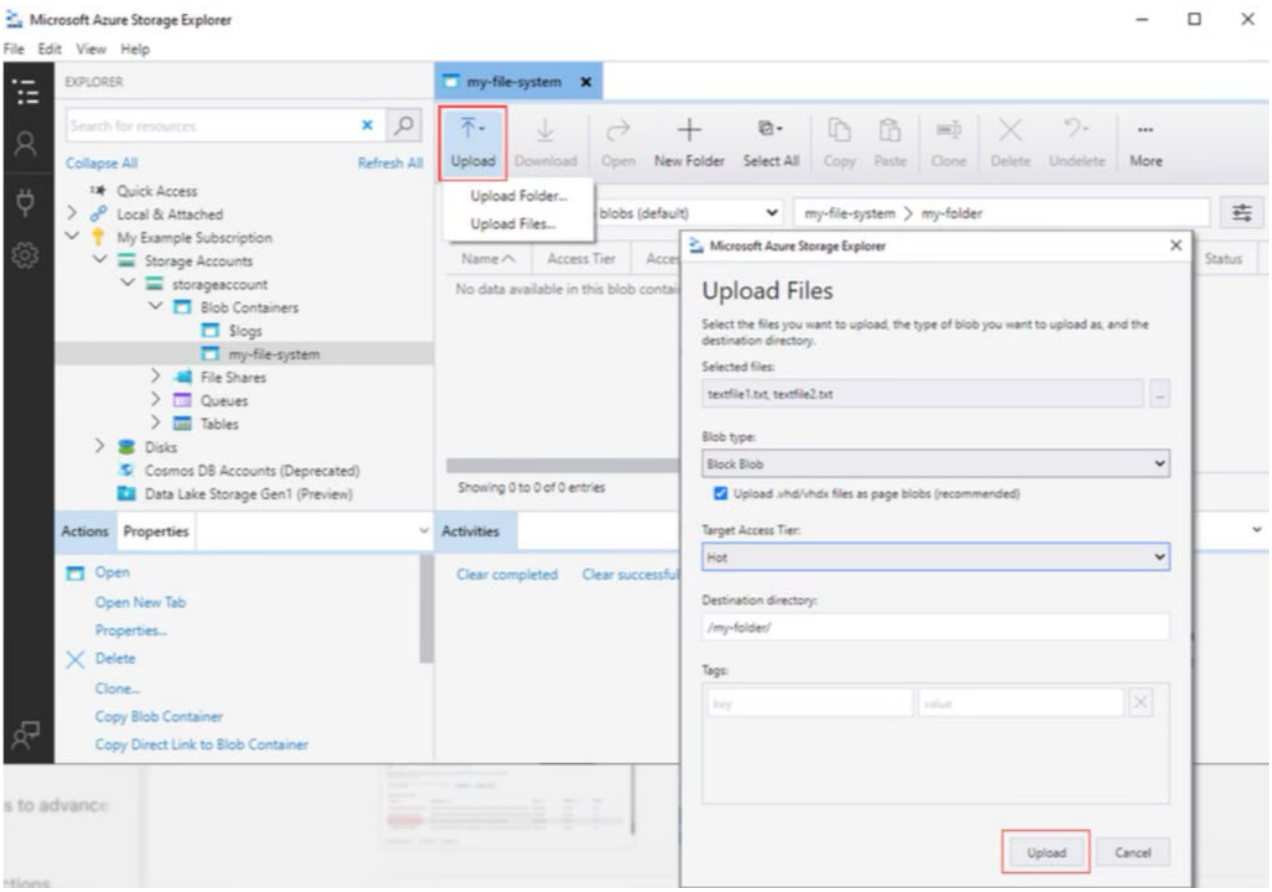
a.Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.

ANS :

Create a standard storage account and enable the Hierarchical Namespace option under the "Advanced" tab. This upgrades the account to ADLS Gen2, enabling directory-level ACLs.

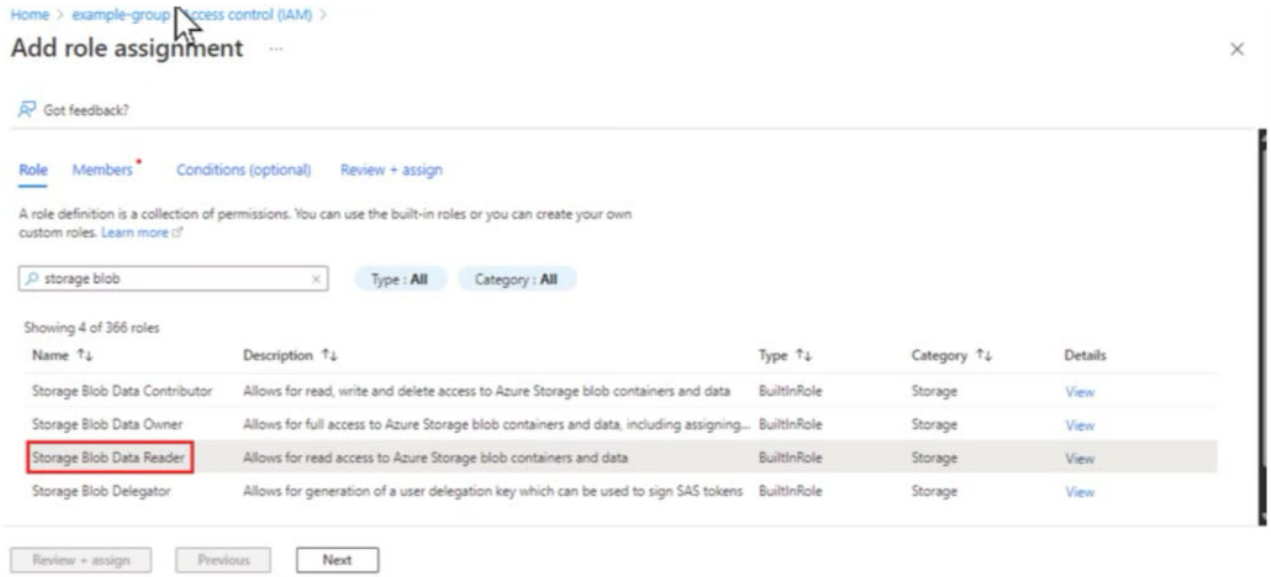


b. Create a filesystem or container and upload a sample dataset.

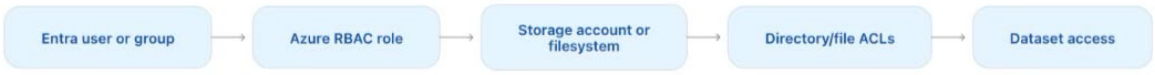


ANS :
Use the Azure CLI or Storage Explorer to create a container (filesystem) and upload a file: az storage fs file upload —account- name <name> --file-system analytics --source data.csv --path raw/data.csv

c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data.
ANS :
Access is managed by assigning roles (like Storage Blob Data Reader) to Microsoft Entra ID users or groups. This ensures only authenticated identities can access the data, rather than relying on shared access keys.



d. Explain how you would verify that unauthorized users cannot access the dataset.
ANS :
Log in as a user who has not been assigned an RBAC role for the storage account. Attempt to list or download the file using Azure Storage Explorer or the CLI; the operation should fail with an Authorization Permission Mismatch error



Azure VM Troubleshooting

If administrators cannot connect to an Azure VM, check its status, network rules, subnet and routes, then use a secure access method.

a.Verify the VM's provisioning and power states.

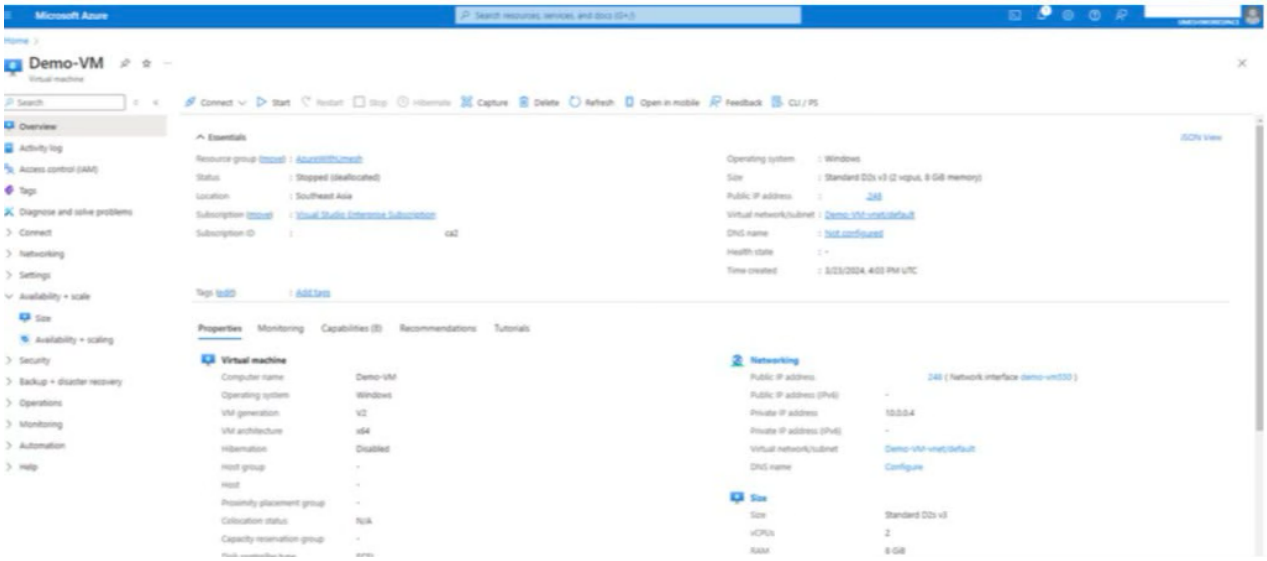
ANS :

In the Azure portal, open Virtual machines → select the VM → **Overview.**

Provisioning state should be Succeeded. A failed provisioning state can mean the VM deployment or a recent configuration change did not complete.

Power state should be Running. If it is stopped or deallocated, start it and retry.

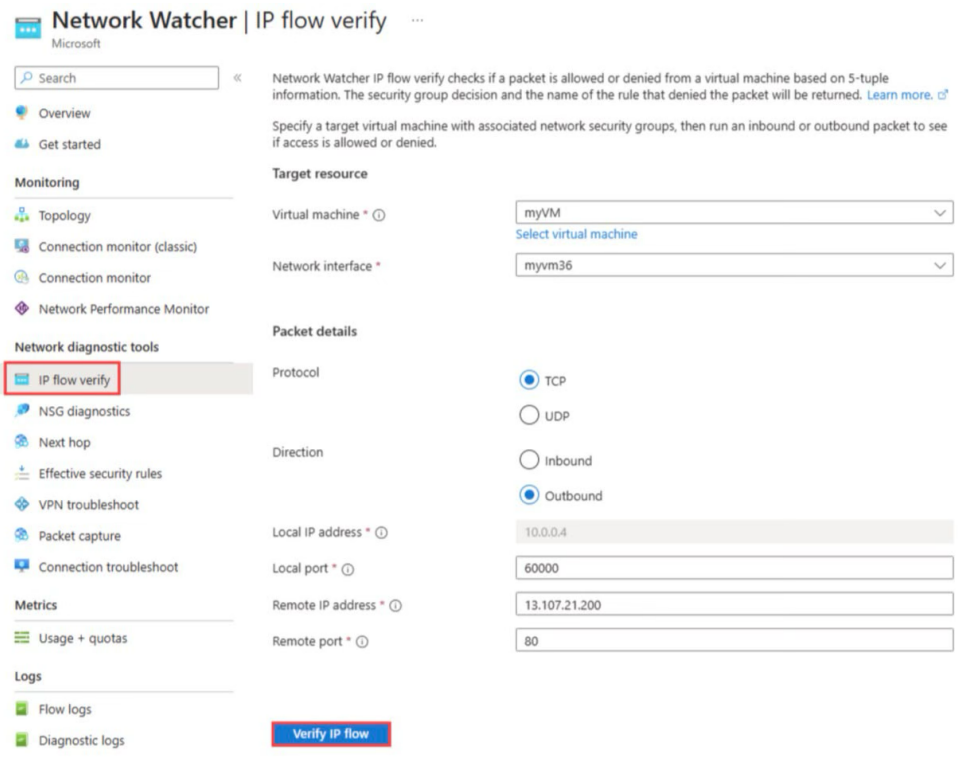
Check Activity log for recent VM, NIC, or network changes that may explain the problem.



b.Explain how Network Security Group rules can prevent connectivity.

ANS :

An NSG can block a connection if it lacks an applicable allow rule or has a higher-priority deny rule. Check NSGs attached to both the subnet and the VM's network interface.



For example, an administrator connecting over SSH needs an inbound TCP rule for port 22; RDP typically uses TCP port 3389. Confirm the rule's source, destination, protocol, port, and priority. Restrict the source to the administrator's known IP range; don't open the port to Any or 0.0.0.0/0

c. Explain how subnet and routing configuration affect connectivity.
ANS :
Confirm the VM's NIC is attached to the intended subnet and that the subnet has the expected address range and connectivity. Inspect the subnet's route table and effective routes: a user-defined route or network appliance may send traffic somewhere unexpected or prevent the return path. Also check peering, VPN, or ExpressRoute connectivity if administrators connect from another network.

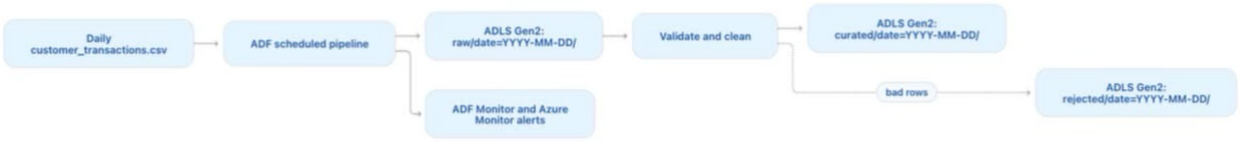
d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.
ANS :
Prefer Azure Bastion for interactive SSH or RDP access. Bastion connects through the Azure portal and does not require a public IP on the VM or
inbound SSH/RDP access from the internet. Restrict Bastion access to authorized administrators.For diagnosis, use Network Watcher →Connection troubleshoot to test connectivity from a source to the VM, and IP flow verify to see whether an NSG rule allows or denies traffic. If available for your environment, use just-in-time VM access to open a tightly scoped management port temporarily, with a limited source IP and duration

Azure Data Integration

Use Azure Data Factory (ADF) to orchestrate ingestion, Azure Data Lake Storage Gen2 (ADLS Gen2) as the landing and curated storage, and an ADF Mapping Data Flow to validate and clean the CSV

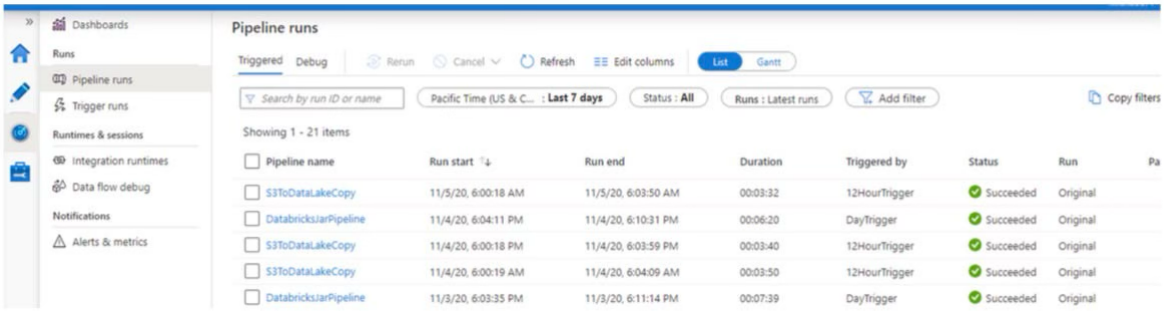
a. Orchestrate data movement

Create an ADF pipeline with a daily schedule trigger. Use a Copy Data activity to move the CSV, then a Mapping Data Flow activity for validation and cleanup.



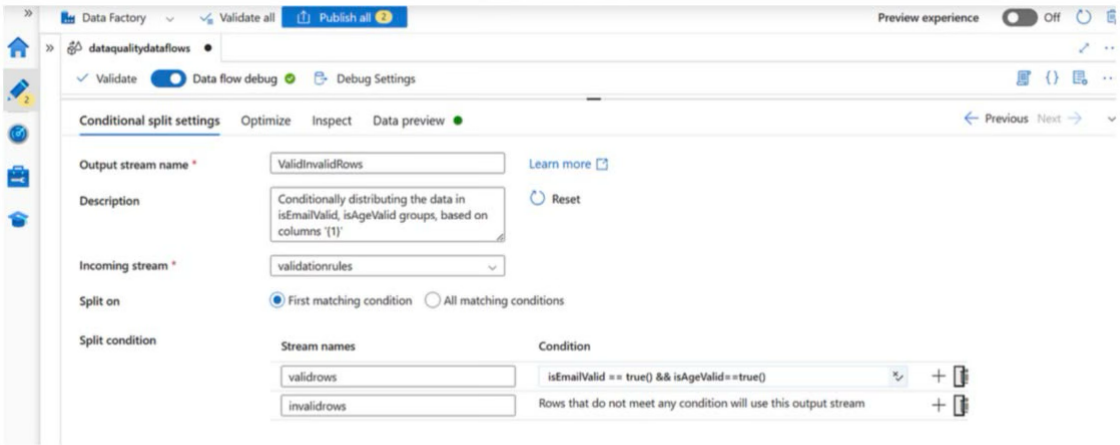
b. Copy the CSV to ADLS Gen2

In ADF Studio, create linked services for the source location and ADLS Gen2, then datasets for the CSV and lake destination. Configure the Copy activity to read the incoming file and write it to the lake. Use Managed
Identity for ADF's access to ADLS, granting it only the required data role at the narrowest suitable scope.



c. Organize by ingestion date

Parameterize the destination folder path and use the pipeline run date,
for example: Keep the original file in raw for traceability; write cleaned data to a separate curated path.



d. Handle malformed records and missing values
In a Mapping Data Flow, validate column types and required fields. Send valid rows to curated and invalid rows to a rejected folder with an error reason. For missing values, apply agreed rules: reject rows missing required fields such as transaction ID, or fill optional fields with a documented default. ADF Copy activity can also skip and log incompatible rows, but a separate rejected-data path makes review and correction easier. ADF Copy activity fault tolerance · Mapping Data Flow Assert transformation

e. Monitor execution and validate completeness
Use ADF's Monitor view to inspect pipeline and activity status, duration, and errors. Set an Azure Monitor alert for failed pipeline runs. Compare source and destination row counts; confirm:Also check that the expected date folder and file exist, and review rejected rows and Copy activity logs before marking the daily load complete. Monitor Azure Data Factor